

4.01-hpi-unos-example

September 2, 2024

1 PRiSM Example: UNOS Heart Transplant Data

This notebook demonstrates the PRiSM (Partial Responses in Structured Models) method, which is designed to transform black-box classifiers into inherently interpretable models without compromising predictive performance. This approach is particularly valuable in high-stakes applications where understanding the model’s decision-making process is crucial.

The method addresses a key challenge in machine learning: the trade-off between model complexity and interpretability. It allows us to create models that are both accurate and transparent. In this example, we’ll use the UNOS (United Network for Organ Sharing) heart transplant dataset to illustrate the PRiSM method.

Here, the target outcome we are modelling is “oneyearmort”, which refers to:

- 0: a patient who survived at least one year after heart transplant
- 1: a patient who died within one year of heart transplant

We would like to use the available features to model the likelihood of one-year mortality after heart transplantation, and understand how each feature contributes to that likelihood.

```
[1]: %load_ext autoreload
      %autoreload 2

      import numpy as np
      import pandas as pd
      import torch
      import matplotlib.pyplot as plt
      import seaborn as sns

      from prism.config import PROCESSED_DATA_DIR, MODELS_DIR
      from prism.preprocessing import normalize, feature_summary, plot_feature_histograms
      from prism.maskedmlp import train_mlp_batched, train_maskedmlp
      from prism.partial_responses import partial_responses
      from prism.response_plot import plot_partial_responses
      from prism.nomogram import nomogram, display_nomograms_side_by_side
      from prism.lasso import lasso
      from prism.device_tools import get_device, get_num_cpu_workers
      from prism.metrics import evaluate_model_performance, compare_model_performance
```

```
2024-09-02 13:38:07.580 | INFO |
prism.config:<module>:11 - PROJ_ROOT path is:
C:\Users\localuser\PRiSM\prism_github
```

```
[2]: %reload_ext autoreload
```

1.1 1. Setup and Imports

First, we need to import the necessary libraries and set up our environment.

We set up our computational device (CPU or GPU if available) and set a random seed for reproducibility.

We'll use the 'dirac' method for partial responses, which is one of two methods (the other being 'lebesgue') for calculating partial responses. The Dirac method calculates the effect of each feature individually, which can be computationally efficient but may miss some complex interactions.

We set up our computational device (CPU or GPU if available) and set a random seed for reproducibility.

Two methods, 'lebesgue' and 'dirac', can be used to calculate partial responses. The Lebesgue method uses an average over the predicted surface, which can be more stable for some datasets, but more computationally intensive. If you're working with a bigger dataset, or low compute resources, starting with 'dirac' may be better.

```
[169]: # Set device to 'cpu', or GPU ('cuda' for NVIDIA, 'mps' for Apple Silicon)
# By default, we select the best available device
device = get_device()
print(f"Using device: {device}")

# Set random seed for reproducibility
random_seed = 257
np.random.seed(random_seed)
torch.manual_seed(random_seed)

# Set method and other parameters
partial_response_method = 'dirac'
SAVE_METRICS = True

# TODO: setup model saving.
SAVE_MODELS = False
```

Using device: cuda

For multithreading compute-intensive tasks, the recommended number of workers for GPU processing is 1, otherwise for CPU stick to the default (number of logical cores - 1). However, depending on your hardware, you may want to experiment.

```
[170]: if device != 'cpu':
        max_workers = 1
```

```

else:
    max_workers = get_num_cpu_workers()
print(f"max_workers = {max_workers}")

```

max_workers = 1

1.2 2. Load and Preprocess Data

Next, we load the UNOS heart transplant data and perform necessary preprocessing steps.

```

[171]: # Load data
data_train = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_train.
↳csv'))
data_test = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_test.
↳csv'))
data_val = pd.read_csv(PROCESSED_DATA_DIR.joinpath('imputed_dataset1_val.csv'))

# Drop id column
data_train.drop('trr_id_code', axis=1, inplace=True)
data_test.drop('trr_id_code', axis=1, inplace=True)
data_val.drop('trr_id_code', axis=1, inplace=True)

target_col = 'oneyearmort'

X_train = data_train.drop(target_col, axis=1)
y_train = data_train[target_col]

X_test = data_test.drop(target_col, axis=1)
y_test = data_test[target_col]

X_val = data_val.drop(target_col, axis=1)
y_val = data_val[target_col]

# Normalize the data
X_train_normalized, X_test_normalized = normalize(X_train, X_test)
X_val_normalized = normalize(X_val)

# Convert to tensors
X_train_tensor = torch.tensor(X_train_normalized.values, dtype=torch.float32,
↳device=device)
X_test_tensor = torch.tensor(X_test_normalized.values, dtype=torch.float32,
↳device=device)
X_val_tensor = torch.tensor(X_val_normalized.values, dtype=torch.float32,
↳device=device)

y_train_tensor = torch.tensor(y_train.values, dtype=torch.float32,
↳device=device)

```

```

y_test_tensor = torch.tensor(y_test.values, dtype=torch.float32, device=device)
y_val_tensor = torch.tensor(y_val.values, dtype=torch.float32, device=device)

feature_names = [
    'don age',
    'don isch time min',
    'rec age',
    'rec creat',
    'rec infect 2wk',
    'rec vent',
    'rec sex',
    'tx year',
    'ICM',
    'NICM',
    'prior tx'
]

```

1.3 3. Data Overview

Before building our models, let's get an overview of the data.

```

[172]: print("Shape of training data:", X_train.shape)
print("Shape of testing data:", X_test.shape)
print("Shape of validation data:", X_val.shape)
print(f"\nClass distribution in training set ({X_train['tx_year'].min()} - {X_train['tx_year'].max()}):")
print(y_train.value_counts(normalize=True))
print(f"\nClass distribution in test set ({X_test['tx_year'].min()} - {X_test['tx_year'].max()}):")
print(y_test.value_counts(normalize=True))
print(f"\nClass distribution in validation set ({X_val['tx_year'].min()} - {X_val['tx_year'].max()}):")
print(y_val.value_counts(normalize=True))

```

```

Shape of training data: (31315, 11)
Shape of testing data: (6120, 11)
Shape of validation data: (4750, 11)

```

Class distribution in training set (1997 - 2013):

```

oneyearmort
0    0.877024
1    0.122976

```

Name: proportion, dtype: float64

Class distribution in test set (2014 - 2016):

```

oneyearmort
0    0.89281
1    0.10719

```

Name: proportion, dtype: float64

Class distribution in validation set (2017 - 2018):

oneyearmort

0 0.900842

1 0.099158

Name: proportion, dtype: float64

```
[173]: feature_stats = feature_summary(X_train, categorical_threshold=15)
print("Feature Statistics:")
print(feature_stats)
```

Feature Statistics:

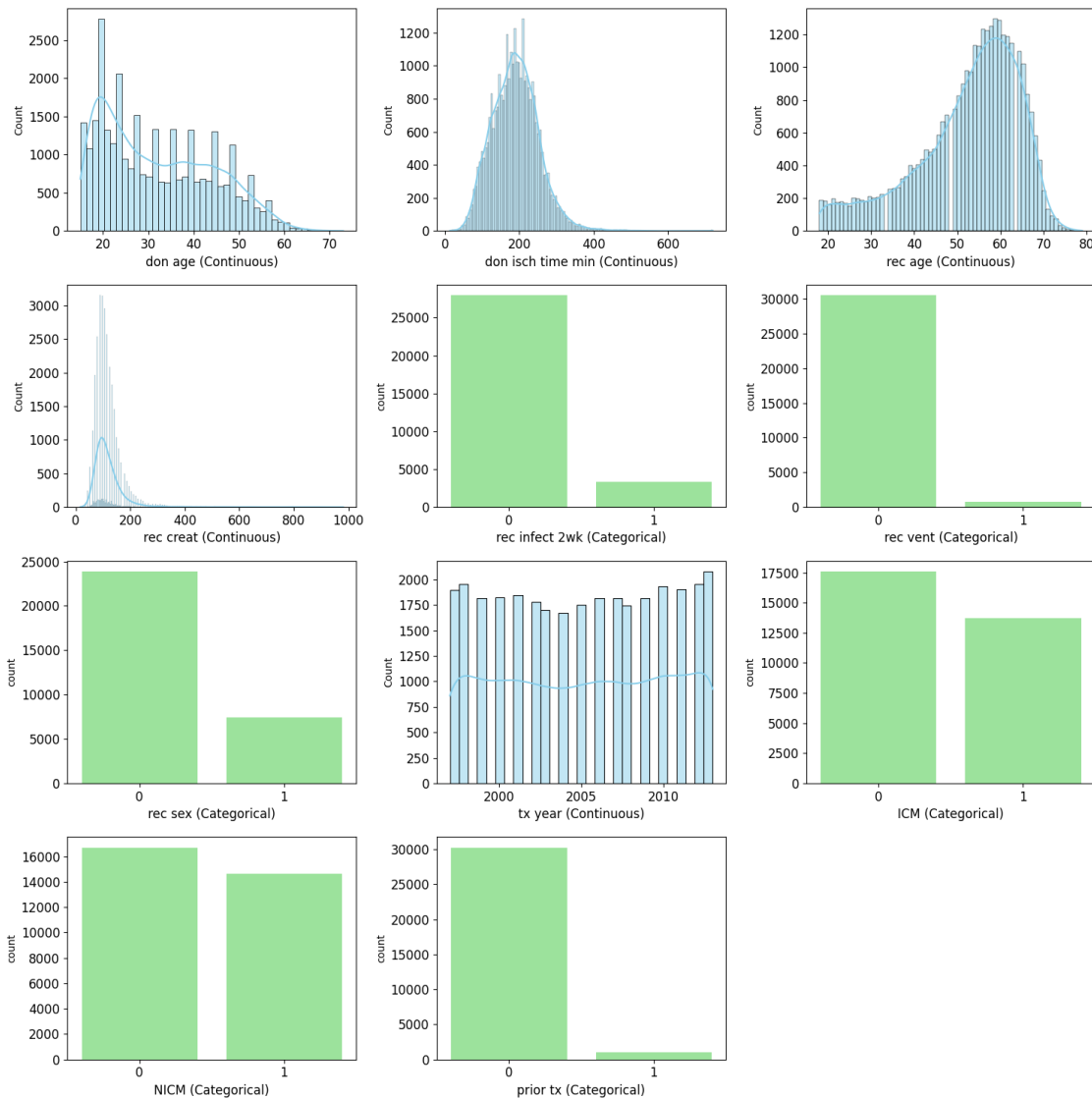
	Data Type	Non-Null Count	Null Count	Mean	\
donage	int64	31315	0	32.091043	
donischemictimeminutes	float64	31315	0	189.745752	
recageyear	int64	31315	0	52.359412	
reccreatininemostrecent	float64	31315	0	119.943712	
recinfections2weeks	int64	31315	0	0.106754	
recventilator	int64	31315	0	0.025770	
recsex	int64	31315	0	0.237043	
tx_year	int64	31315	0	2005.084943	
diagnICM	int64	31315	0	0.437841	
diagnNICM	int64	31315	0	0.466741	
recpriortx	int64	31315	0	0.034424	

	Median	Std Dev	IQR	Min	\
donage	30.00000	12.018143	21.000000	15.000000	
donischemictimeminutes	187.99805	62.573339	82.998050	12.998657	
recageyear	55.00000	12.169868	15.000000	18.000000	
reccreatininemostrecent	106.08000	67.657171	44.200008	16.796000	
recinfections2weeks	0.00000	0.308805	0.000000	0.000000	
recventilator	0.00000	0.158452	0.000000	0.000000	
recsex	0.00000	0.425276	0.000000	0.000000	
tx_year	2005.00000	4.996192	9.000000	1997.000000	
diagnICM	0.00000	0.496129	1.000000	0.000000	
diagnNICM	0.00000	0.498901	1.000000	0.000000	
recpriortx	0.00000	0.182320	0.000000	0.000000	

	Max	Unique Values	Is Categorical
donage	73.00000	59	False
donischemictimeminutes	720.00000	454	False
recageyear	79.00000	62	False
reccreatininemostrecent	981.23999	396	False
recinfections2weeks	1.00000	2	True
recventilator	1.00000	2	True
recsex	1.00000	2	True
tx_year	2013.00000	17	False

diagnICM	1.00000	2	True
diagnNICM	1.00000	2	True
recpriortx	1.00000	2	True

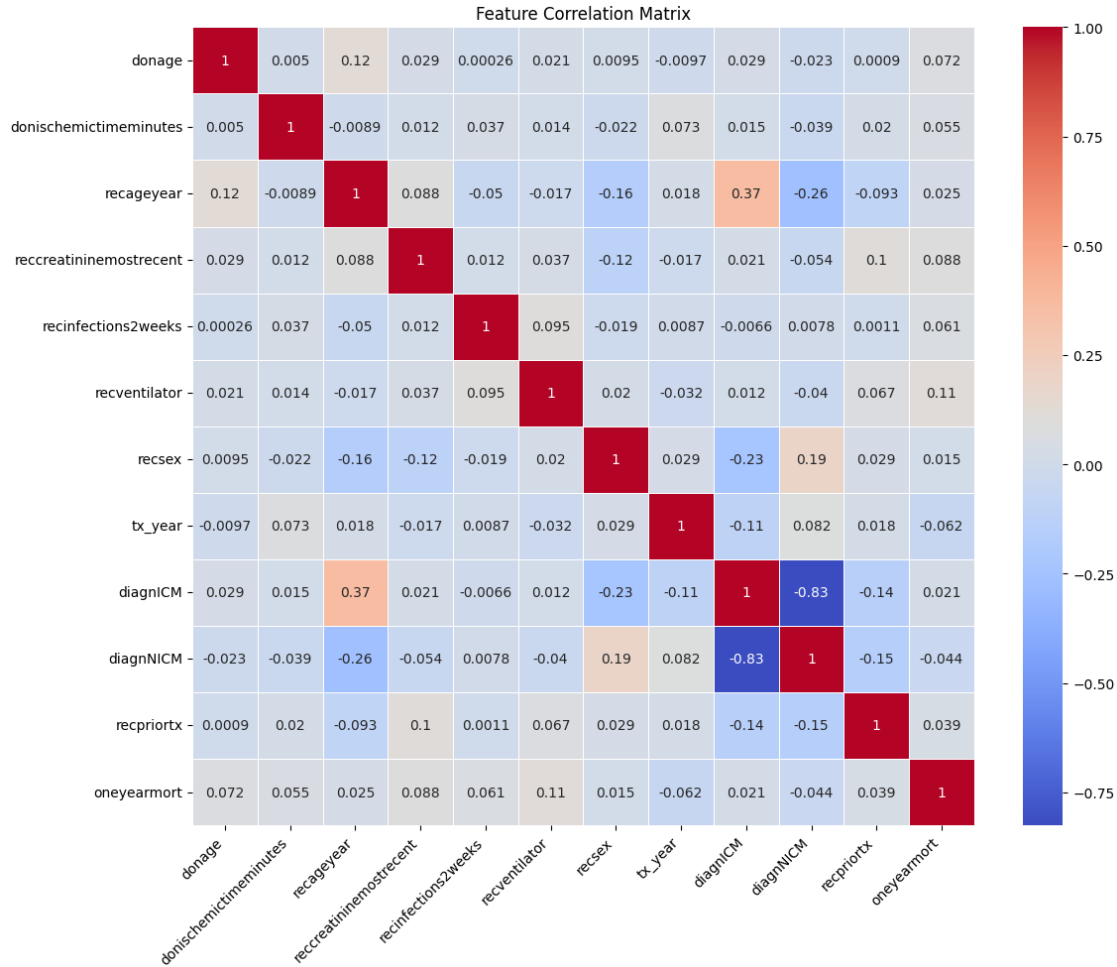
```
[174]: plot_feature_histograms(X_train, feature_stats, figsize=(15,15),
    ↪ feature_names=feature_names)
```



```
[174]: <module 'matplotlib.pyplot' from
    'c:\\Users\\localuser\\PRISM\\prism_github\\venv_prism\\Lib\\site-
    packages\\matplotlib\\pyplot.py'>
```

```
[175]: # Correlation matrix
correlation_matrix = pd.concat([X_train, y_train], axis=1).corr()
```

```
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Feature Correlation Matrix')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



1.4 4. Train Initial Black Box Model (MLP)

We start by training a black box model, in this case, a Multi-Layer Perceptron (MLP) with 10 nodes in the hidden layer. This model serves as our baseline and represents the type of hard-to-interpret model that is often used in machine learning.

```
[176]: mlp_hyperparameters = {
    'n_hidden': 10,
    'weight_decay': 1e-5,
    'lr': 0.001,
```

```

    'patience': 50,
    'tolerance': 0.0001,
    'batch_size': 1024,
    'device': device,
    'seed': 1
}

blackbox_model = train_mlp_batched(X_train_normalized, y_train,
    ↪X_test_normalized, y_test, **mlp_hyperparameters)

```

```

Epoch 0: Train loss 0.7475, Test loss 0.6415
Epoch 1: Train loss 0.6555, Test loss 0.5584
Epoch 2: Train loss 0.5816, Test loss 0.4930
Epoch 3: Train loss 0.5229, Test loss 0.4445
Epoch 4: Train loss 0.4772, Test loss 0.4081
Epoch 5: Train loss 0.4433, Test loss 0.3831
Epoch 6: Train loss 0.4187, Test loss 0.3662
Epoch 7: Train loss 0.4026, Test loss 0.3554
Epoch 8: Train loss 0.3910, Test loss 0.3488
Epoch 9: Train loss 0.3826, Test loss 0.3443
Epoch 10: Train loss 0.3773, Test loss 0.3417
Epoch 11: Train loss 0.3738, Test loss 0.3403
Epoch 12: Train loss 0.3710, Test loss 0.3394
Epoch 13: Train loss 0.3680, Test loss 0.3389
Epoch 14: Train loss 0.3671, Test loss 0.3387
Epoch 15: Train loss 0.3654, Test loss 0.3385
Epoch 16: Train loss 0.3642, Test loss 0.3385
Epoch 17: Train loss 0.3629, Test loss 0.3385
Epoch 18: Train loss 0.3628, Test loss 0.3387
Epoch 19: Train loss 0.3616, Test loss 0.3387
Epoch 20: Train loss 0.3615, Test loss 0.3387
Epoch 21: Train loss 0.3611, Test loss 0.3388
Epoch 22: Train loss 0.3616, Test loss 0.3390
Epoch 23: Train loss 0.3600, Test loss 0.3389
Epoch 24: Train loss 0.3595, Test loss 0.3391
Epoch 25: Train loss 0.3592, Test loss 0.3390
Epoch 26: Train loss 0.3594, Test loss 0.3390
Epoch 27: Train loss 0.3596, Test loss 0.3391
Epoch 28: Train loss 0.3583, Test loss 0.3392
Epoch 29: Train loss 0.3584, Test loss 0.3392
Epoch 30: Train loss 0.3580, Test loss 0.3392
Epoch 31: Train loss 0.3589, Test loss 0.3392
Epoch 32: Train loss 0.3580, Test loss 0.3391
Epoch 33: Train loss 0.3585, Test loss 0.3390
Epoch 34: Train loss 0.3575, Test loss 0.3393
Epoch 35: Train loss 0.3578, Test loss 0.3391
Epoch 36: Train loss 0.3574, Test loss 0.3390
Epoch 37: Train loss 0.3582, Test loss 0.3391

```

Epoch 38: Train loss 0.3574, Test loss 0.3390
Epoch 39: Train loss 0.3582, Test loss 0.3390
Epoch 40: Train loss 0.3575, Test loss 0.3388
Epoch 41: Train loss 0.3570, Test loss 0.3387
Epoch 42: Train loss 0.3573, Test loss 0.3389
Epoch 43: Train loss 0.3572, Test loss 0.3387
Epoch 44: Train loss 0.3572, Test loss 0.3387
Epoch 45: Train loss 0.3574, Test loss 0.3388
Epoch 46: Train loss 0.3570, Test loss 0.3386
Epoch 47: Train loss 0.3565, Test loss 0.3385
Epoch 48: Train loss 0.3564, Test loss 0.3384
Epoch 49: Train loss 0.3561, Test loss 0.3384
Epoch 50: Train loss 0.3567, Test loss 0.3385
Epoch 51: Train loss 0.3563, Test loss 0.3384
Epoch 52: Train loss 0.3560, Test loss 0.3384
Epoch 53: Train loss 0.3568, Test loss 0.3383
Epoch 54: Train loss 0.3562, Test loss 0.3385
Epoch 55: Train loss 0.3558, Test loss 0.3382
Epoch 56: Train loss 0.3566, Test loss 0.3383
Epoch 57: Train loss 0.3566, Test loss 0.3383
Epoch 58: Train loss 0.3564, Test loss 0.3379
Epoch 59: Train loss 0.3561, Test loss 0.3381
Epoch 60: Train loss 0.3560, Test loss 0.3381
Epoch 61: Train loss 0.3558, Test loss 0.3380
Epoch 62: Train loss 0.3553, Test loss 0.3380
Epoch 63: Train loss 0.3558, Test loss 0.3382
Epoch 64: Train loss 0.3556, Test loss 0.3380
Epoch 65: Train loss 0.3558, Test loss 0.3381
Epoch 66: Train loss 0.3559, Test loss 0.3380
Epoch 67: Train loss 0.3557, Test loss 0.3380
Epoch 68: Train loss 0.3558, Test loss 0.3379
Epoch 69: Train loss 0.3555, Test loss 0.3379
Epoch 70: Train loss 0.3557, Test loss 0.3378
Epoch 71: Train loss 0.3559, Test loss 0.3379
Epoch 72: Train loss 0.3553, Test loss 0.3375
Epoch 73: Train loss 0.3564, Test loss 0.3379
Epoch 74: Train loss 0.3555, Test loss 0.3377
Epoch 75: Train loss 0.3557, Test loss 0.3376
Epoch 76: Train loss 0.3558, Test loss 0.3376
Epoch 77: Train loss 0.3558, Test loss 0.3375
Epoch 78: Train loss 0.3555, Test loss 0.3376
Epoch 79: Train loss 0.3554, Test loss 0.3377
Epoch 80: Train loss 0.3554, Test loss 0.3377
Epoch 81: Train loss 0.3556, Test loss 0.3375
Epoch 82: Train loss 0.3554, Test loss 0.3374
Epoch 83: Train loss 0.3559, Test loss 0.3376
Epoch 84: Train loss 0.3548, Test loss 0.3374
Epoch 85: Train loss 0.3550, Test loss 0.3375

Epoch 86: Train loss 0.3560, Test loss 0.3374
Epoch 87: Train loss 0.3546, Test loss 0.3372
Epoch 88: Train loss 0.3550, Test loss 0.3373
Epoch 89: Train loss 0.3549, Test loss 0.3376
Epoch 90: Train loss 0.3551, Test loss 0.3375
Epoch 91: Train loss 0.3552, Test loss 0.3373
Epoch 92: Train loss 0.3550, Test loss 0.3373
Epoch 93: Train loss 0.3553, Test loss 0.3374
Epoch 94: Train loss 0.3549, Test loss 0.3372
Epoch 95: Train loss 0.3549, Test loss 0.3373
Epoch 96: Train loss 0.3546, Test loss 0.3375
Epoch 97: Train loss 0.3550, Test loss 0.3372
Epoch 98: Train loss 0.3545, Test loss 0.3373
Epoch 99: Train loss 0.3550, Test loss 0.3373
Epoch 100: Train loss 0.3551, Test loss 0.3372
Epoch 101: Train loss 0.3548, Test loss 0.3370
Epoch 102: Train loss 0.3547, Test loss 0.3371
Epoch 103: Train loss 0.3545, Test loss 0.3370
Epoch 104: Train loss 0.3555, Test loss 0.3374
Epoch 105: Train loss 0.3552, Test loss 0.3371
Epoch 106: Train loss 0.3549, Test loss 0.3370
Epoch 107: Train loss 0.3551, Test loss 0.3371
Epoch 108: Train loss 0.3551, Test loss 0.3371
Epoch 109: Train loss 0.3547, Test loss 0.3370
Epoch 110: Train loss 0.3548, Test loss 0.3371
Epoch 111: Train loss 0.3550, Test loss 0.3372
Epoch 112: Train loss 0.3548, Test loss 0.3368
Epoch 113: Train loss 0.3549, Test loss 0.3368
Epoch 114: Train loss 0.3542, Test loss 0.3367
Epoch 115: Train loss 0.3549, Test loss 0.3371
Epoch 116: Train loss 0.3545, Test loss 0.3368
Epoch 117: Train loss 0.3543, Test loss 0.3367
Epoch 118: Train loss 0.3546, Test loss 0.3368
Epoch 119: Train loss 0.3549, Test loss 0.3368
Epoch 120: Train loss 0.3547, Test loss 0.3369
Epoch 121: Train loss 0.3541, Test loss 0.3366
Epoch 122: Train loss 0.3541, Test loss 0.3368
Epoch 123: Train loss 0.3546, Test loss 0.3366
Epoch 124: Train loss 0.3544, Test loss 0.3369
Epoch 125: Train loss 0.3544, Test loss 0.3368
Epoch 126: Train loss 0.3546, Test loss 0.3366
Epoch 127: Train loss 0.3544, Test loss 0.3368
Epoch 128: Train loss 0.3550, Test loss 0.3368
Epoch 129: Train loss 0.3548, Test loss 0.3365
Epoch 130: Train loss 0.3543, Test loss 0.3366
Epoch 131: Train loss 0.3548, Test loss 0.3369
Epoch 132: Train loss 0.3543, Test loss 0.3365
Epoch 133: Train loss 0.3550, Test loss 0.3367

Epoch 134: Train loss 0.3547, Test loss 0.3365
Epoch 135: Train loss 0.3540, Test loss 0.3366
Epoch 136: Train loss 0.3541, Test loss 0.3366
Epoch 137: Train loss 0.3544, Test loss 0.3365
Epoch 138: Train loss 0.3547, Test loss 0.3366
Epoch 139: Train loss 0.3546, Test loss 0.3365
Epoch 140: Train loss 0.3542, Test loss 0.3365
Epoch 141: Train loss 0.3549, Test loss 0.3368
Epoch 142: Train loss 0.3557, Test loss 0.3366
Epoch 143: Train loss 0.3545, Test loss 0.3364
Epoch 144: Train loss 0.3544, Test loss 0.3363
Epoch 145: Train loss 0.3546, Test loss 0.3366
Epoch 146: Train loss 0.3539, Test loss 0.3364
Epoch 147: Train loss 0.3545, Test loss 0.3364
Epoch 148: Train loss 0.3547, Test loss 0.3363
Epoch 149: Train loss 0.3549, Test loss 0.3365
Epoch 150: Train loss 0.3539, Test loss 0.3363
Epoch 151: Train loss 0.3540, Test loss 0.3364
Epoch 152: Train loss 0.3552, Test loss 0.3368
Epoch 153: Train loss 0.3544, Test loss 0.3362
Epoch 154: Train loss 0.3540, Test loss 0.3364
Epoch 155: Train loss 0.3543, Test loss 0.3362
Epoch 156: Train loss 0.3547, Test loss 0.3367
Epoch 157: Train loss 0.3544, Test loss 0.3363
Epoch 158: Train loss 0.3543, Test loss 0.3362
Epoch 159: Train loss 0.3545, Test loss 0.3364
Epoch 160: Train loss 0.3541, Test loss 0.3363
Epoch 161: Train loss 0.3547, Test loss 0.3365
Epoch 162: Train loss 0.3549, Test loss 0.3365
Epoch 163: Train loss 0.3536, Test loss 0.3363
Epoch 164: Train loss 0.3543, Test loss 0.3366
Epoch 165: Train loss 0.3542, Test loss 0.3362
Epoch 166: Train loss 0.3539, Test loss 0.3362
Epoch 167: Train loss 0.3539, Test loss 0.3364
Epoch 168: Train loss 0.3536, Test loss 0.3364
Epoch 169: Train loss 0.3550, Test loss 0.3364
Epoch 170: Train loss 0.3543, Test loss 0.3362
Epoch 171: Train loss 0.3547, Test loss 0.3366
Epoch 172: Train loss 0.3538, Test loss 0.3361
Epoch 173: Train loss 0.3533, Test loss 0.3361
Epoch 174: Train loss 0.3547, Test loss 0.3364
Epoch 175: Train loss 0.3540, Test loss 0.3361
Epoch 176: Train loss 0.3542, Test loss 0.3364
Epoch 177: Train loss 0.3544, Test loss 0.3364
Epoch 178: Train loss 0.3548, Test loss 0.3361
Epoch 179: Train loss 0.3540, Test loss 0.3362
Epoch 180: Train loss 0.3538, Test loss 0.3360
Epoch 181: Train loss 0.3541, Test loss 0.3363

Epoch 182: Train loss 0.3538, Test loss 0.3362
Epoch 183: Train loss 0.3544, Test loss 0.3365
Epoch 184: Train loss 0.3540, Test loss 0.3362
Epoch 185: Train loss 0.3543, Test loss 0.3362
Epoch 186: Train loss 0.3544, Test loss 0.3360
Epoch 187: Train loss 0.3534, Test loss 0.3359
Epoch 188: Train loss 0.3539, Test loss 0.3361
Epoch 189: Train loss 0.3537, Test loss 0.3362
Epoch 190: Train loss 0.3539, Test loss 0.3363
Epoch 191: Train loss 0.3543, Test loss 0.3360
Epoch 192: Train loss 0.3546, Test loss 0.3360
Epoch 193: Train loss 0.3543, Test loss 0.3362
Epoch 194: Train loss 0.3531, Test loss 0.3360
Epoch 195: Train loss 0.3543, Test loss 0.3364
Epoch 196: Train loss 0.3539, Test loss 0.3363
Epoch 197: Train loss 0.3543, Test loss 0.3362
Epoch 198: Train loss 0.3536, Test loss 0.3362
Epoch 199: Train loss 0.3544, Test loss 0.3362
Epoch 200: Train loss 0.3539, Test loss 0.3361
Epoch 201: Train loss 0.3540, Test loss 0.3361
Epoch 202: Train loss 0.3540, Test loss 0.3361
Epoch 203: Train loss 0.3537, Test loss 0.3361
Epoch 204: Train loss 0.3544, Test loss 0.3361
Epoch 205: Train loss 0.3536, Test loss 0.3360
Epoch 206: Train loss 0.3537, Test loss 0.3361
Epoch 207: Train loss 0.3543, Test loss 0.3362
Epoch 208: Train loss 0.3540, Test loss 0.3360
Epoch 209: Train loss 0.3539, Test loss 0.3359
Epoch 210: Train loss 0.3543, Test loss 0.3363
Epoch 211: Train loss 0.3540, Test loss 0.3361
Epoch 212: Train loss 0.3548, Test loss 0.3362
Epoch 213: Train loss 0.3539, Test loss 0.3361
Epoch 214: Train loss 0.3538, Test loss 0.3362
Epoch 215: Train loss 0.3536, Test loss 0.3362
Epoch 216: Train loss 0.3539, Test loss 0.3362
Epoch 217: Train loss 0.3534, Test loss 0.3362
Epoch 218: Train loss 0.3537, Test loss 0.3363
Epoch 219: Train loss 0.3542, Test loss 0.3361
Epoch 220: Train loss 0.3536, Test loss 0.3360
Epoch 221: Train loss 0.3540, Test loss 0.3362
Epoch 222: Train loss 0.3540, Test loss 0.3361
Epoch 223: Train loss 0.3540, Test loss 0.3360
Epoch 224: Train loss 0.3549, Test loss 0.3364
Epoch 225: Train loss 0.3538, Test loss 0.3362
Epoch 226: Train loss 0.3543, Test loss 0.3361
Epoch 227: Train loss 0.3537, Test loss 0.3362
Epoch 228: Train loss 0.3536, Test loss 0.3360
Epoch 229: Train loss 0.3535, Test loss 0.3363

```
Epoch 230: Train loss 0.3542, Test loss 0.3362
Epoch 231: Train loss 0.3537, Test loss 0.3360
Epoch 232: Train loss 0.3535, Test loss 0.3361
Epoch 233: Train loss 0.3539, Test loss 0.3362
Epoch 234: Train loss 0.3533, Test loss 0.3361
Epoch 235: Train loss 0.3539, Test loss 0.3361
Epoch 236: Train loss 0.3536, Test loss 0.3363
Stopping early at epoch 187
```

```
[177]: y_pred_train_blackbox = blackbox_model.predict(X_train_tensor)
y_pred_test_blackbox = blackbox_model.predict(X_test_tensor)

# Evaluate MLP
results_mlp_train = evaluate_model_performance(y_train, y_pred_train_blackbox,
↪y_train, title="MLP - training data")
results_mlp_test = evaluate_model_performance(y_test, y_pred_test_blackbox,
↪y_train, title="MLP - test data")
```

---- Model Performance Metrics MLP - training data ----

Threshold (prevalence) set to: 0.123

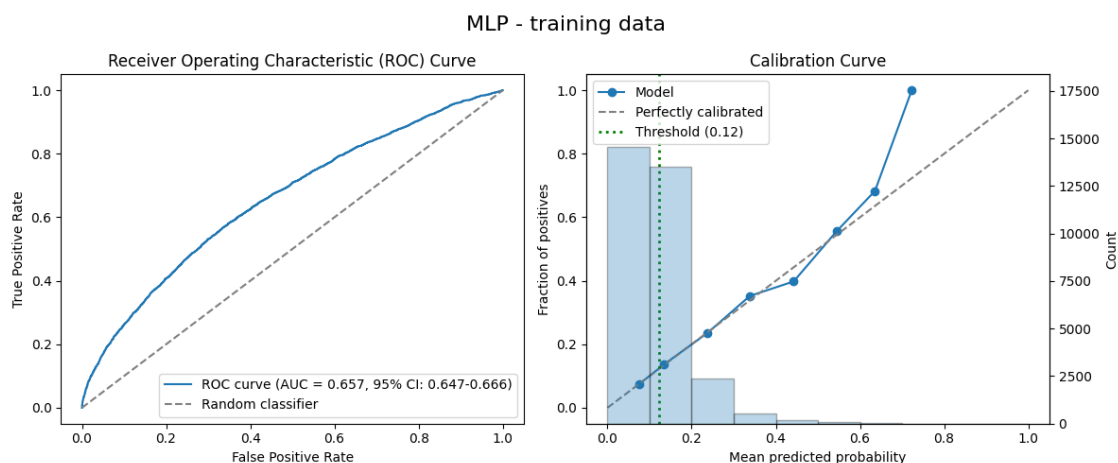
AUROC: 0.657 (95% CI: 0.647-0.666) (Area Under the Receiver Operating Characteristic curve)

Accuracy: 0.665 (Proportion of correct predictions)

Precision: 0.194 (Proportion of true positives among positive predictions, aka PPV)

Recall: 0.548 (Proportion of true positives among actual positives, aka sensitivity)

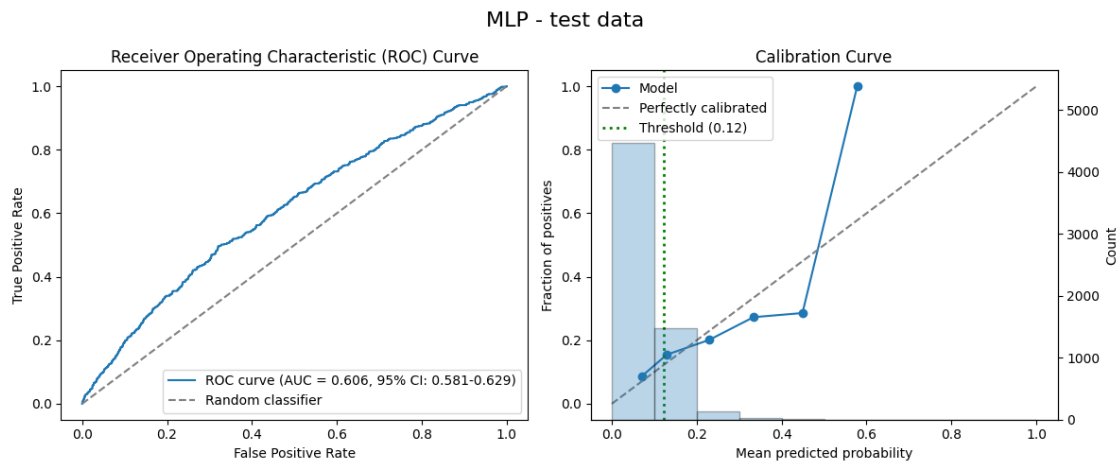
F1 Score: 0.287 (Harmonic mean of precision and recall)



```

---- Model Performance Metrics MLP - test data ----
Threshold (prevalence) set to: 0.123
AUROC:          0.606 (95% CI: 0.581-0.629) (Area Under the Receiver Operating
Characteristic curve)
Accuracy:       0.801 (Proportion of correct predictions)
Precision:      0.177 (Proportion of true positives among positive predictions,
aka PPV)
Recall:         0.235 (Proportion of true positives among actual positives, aka
sensitivity)
F1 Score:      0.202 (Harmonic mean of precision and recall)
-----

```



1.5 5. Calculate Partial Responses

Now we calculate partial responses for our black box model. This is a key step in the PRiSM method. Partial responses help us understand how individual features or pairs of features contribute to the model's predictions, making the black box model more interpretable.

```

[178]: partial_responses_params = {
        'x_train': X_train_tensor,
        'method': partial_response_method,
        'device': device,
        'batch_size': 512,
        'max_workers': max_workers
    }

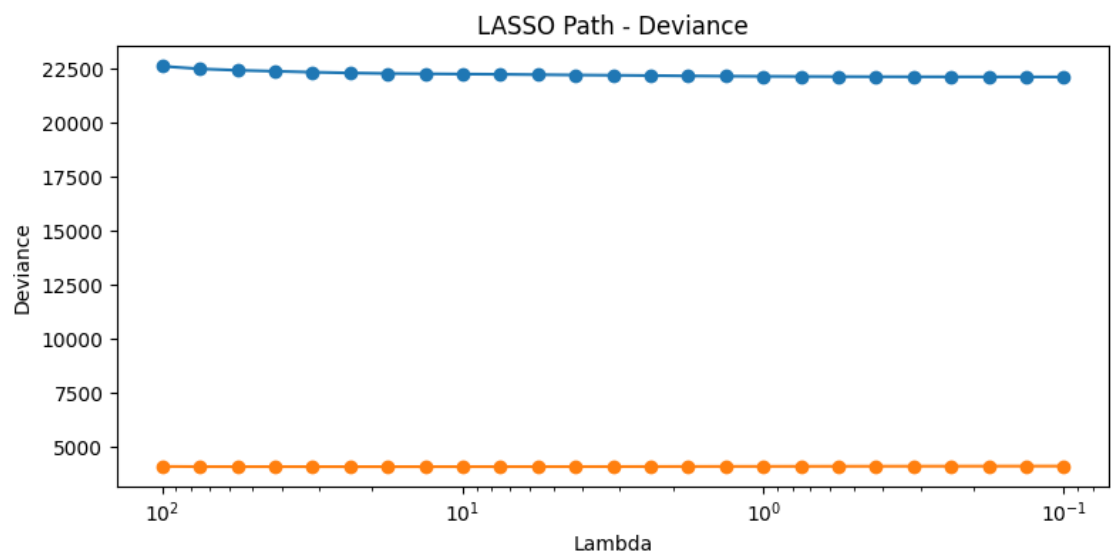
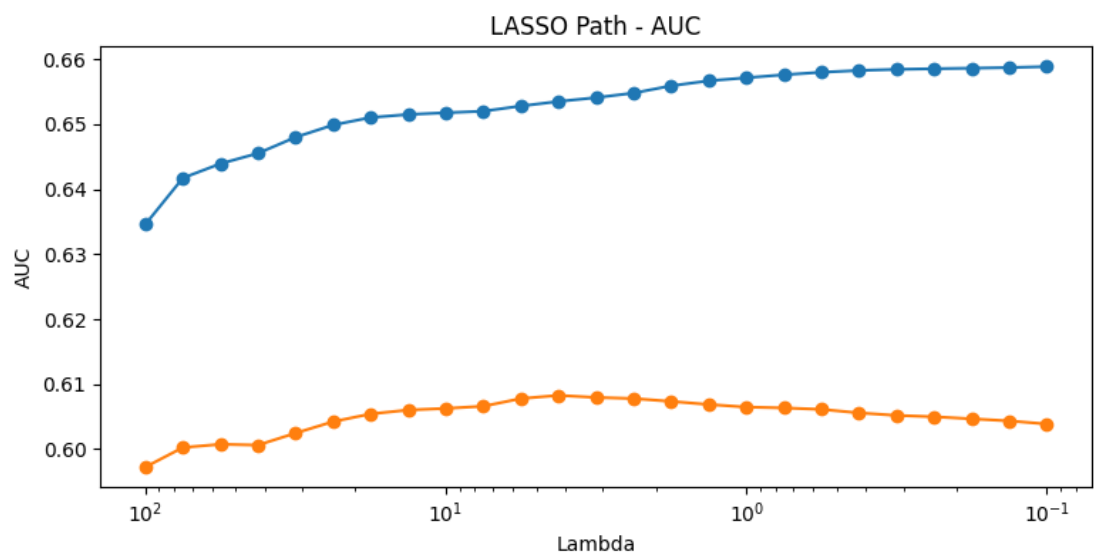
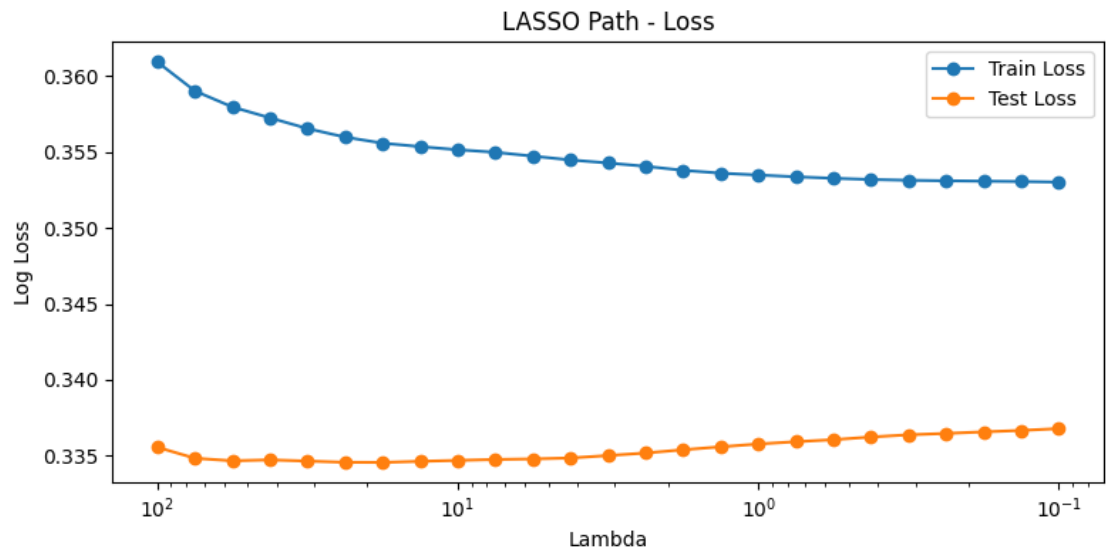
partial_responses_train = partial_responses(X_train_tensor, blackbox_model,
    ↪**partial_responses_params)
partial_responses_test = partial_responses(X_test_tensor, blackbox_model,
    ↪**partial_responses_params)

```

1.6 6. Perform LASSO on Partial Responses

We use LASSO (Least Absolute Shrinkage and Selection Operator) to select the most consequential partial responses. This step helps us identify which features or feature interactions contribute most to the model's predictions. A range of regularization strengths (λ) are used, and for each one a logistic regression model with LASSO regularization is trained and evaluated. Based on the results, we can choose the most suitable λ , balancing model performance against the total number of selected features.

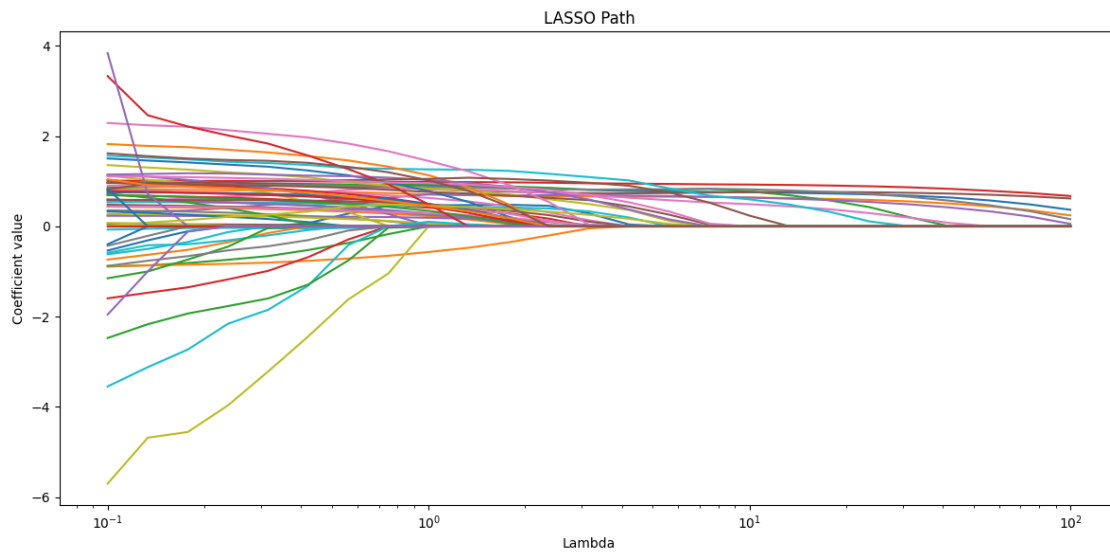
```
[179]: lasso_results = lasso(  
    partial_responses_train,  
    partial_responses_test,  
    y_train,  
    y_test,  
    feature_names=feature_names,  
    nlambda=25,  
    min_lambda=0.1,  
    max_lambda=100,  
    batch_size=2,  
    seed=random_seed  
)
```



<IPython.core.display.HTML object>

All fits converged successfully.

```
[180]: lasso_results.plot_lambda_path()
```



```
[181]: lasso_results.select_lambda_max_test_auc()

# Alternatively, manually select lambda as needed
# lasso_results.select_lambda(11)
```

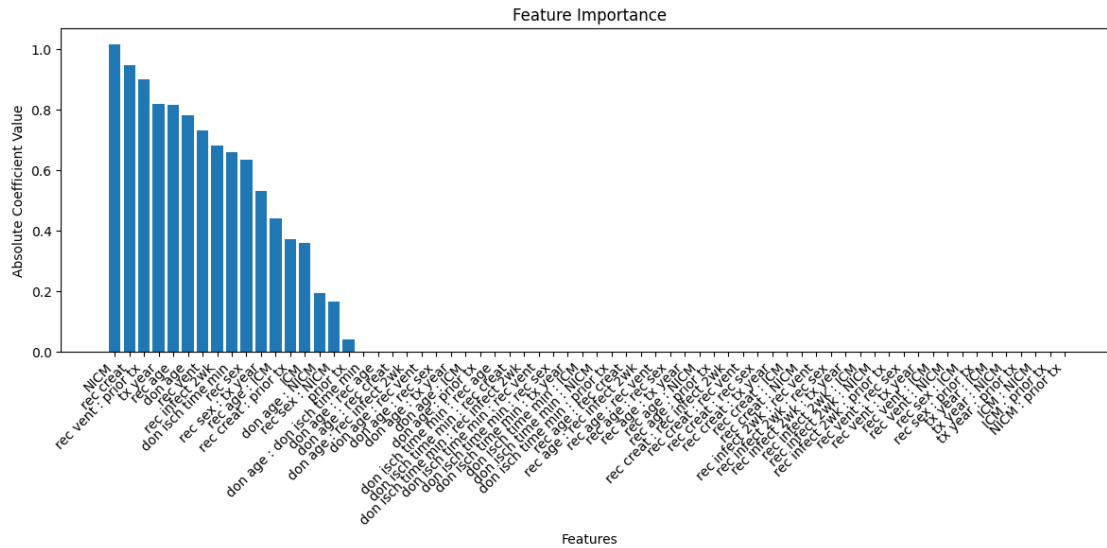
Selected lambda index: 11

Selected lambda value: 4.2170

Corresponding test AUC: 0.6083

```
[181]: 11
```

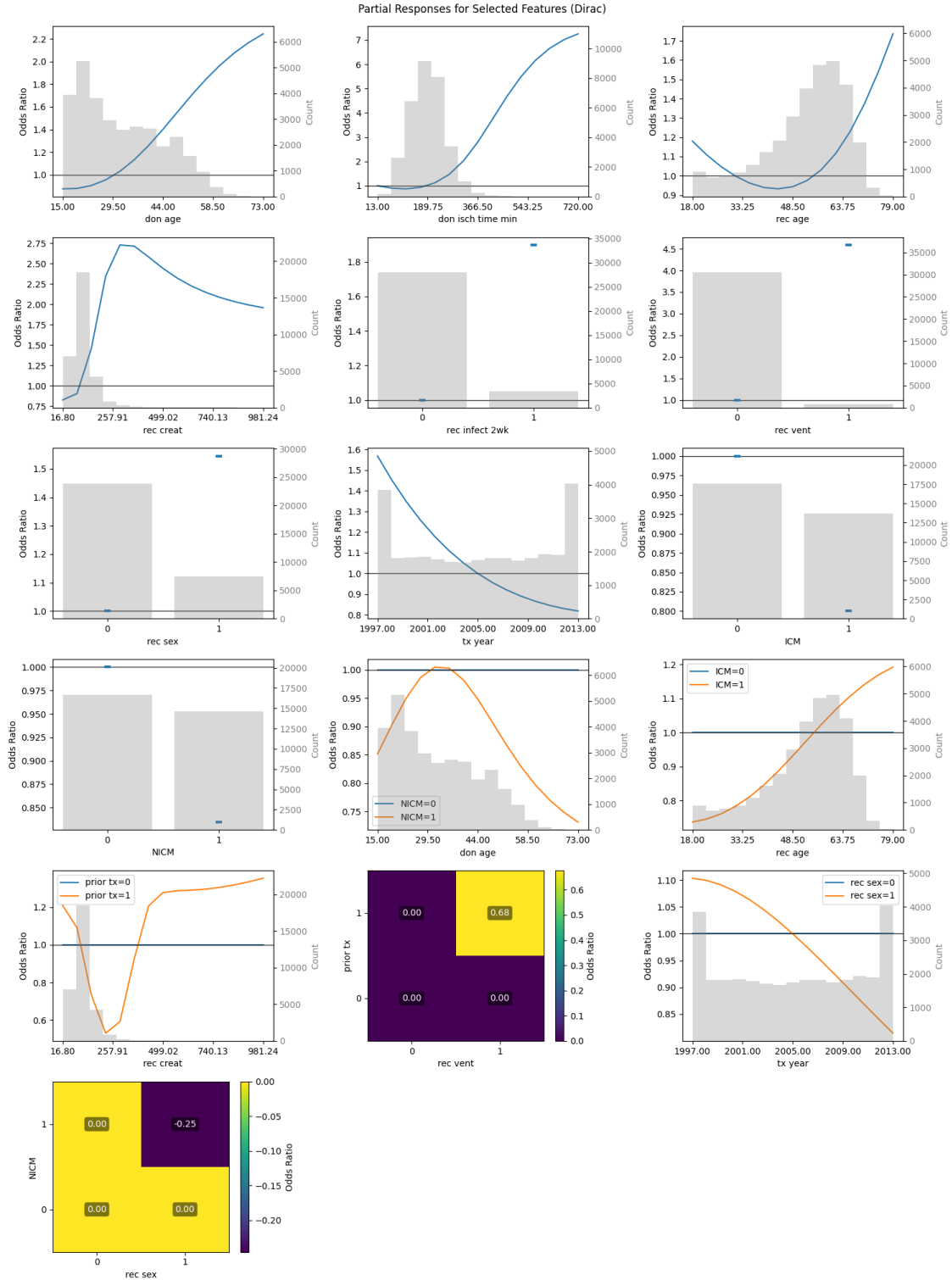
```
[182]: lasso_results.plot_feature_importance()
```



1.7 7. View Selected Partial Responses and Generate Nomogram

First, we visualize the contribution of each partial response selected with LASSO to the target value.

```
[183]: fig_blackbox = plot_partial_responses(
    lasso_results,
    X_train_tensor,
    X_train.median().values,
    X_train.std().values,
    blackbox_model,
    n_steps=15,
    sd_scale=2,
    method=partial_response_method,
    device=device,
    categorical_threshold=15,
    subtract_univariate=True,
    figsize=(15,20),
    show_fig=True,
    return_fig=True,
    use_odds_ratio=True
)
```



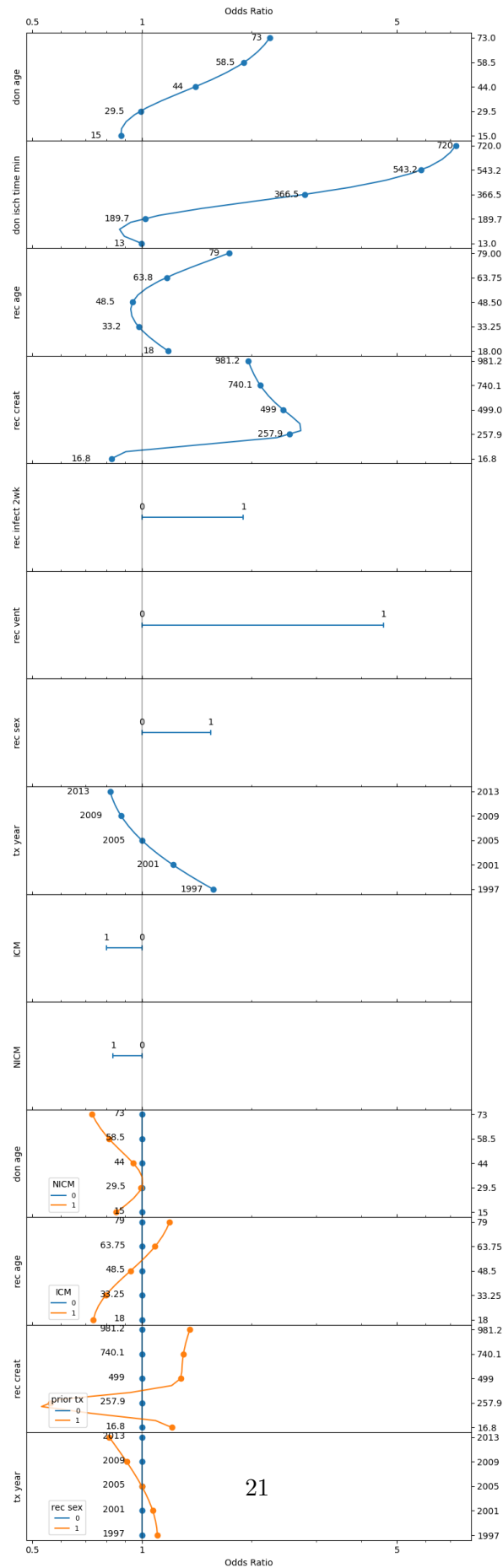
We can also visualize the contribution of each partial response using a nomogram. Nomograms are particularly useful in clinical settings as they provide an intuitive visual representation of a model's

decision-making process.

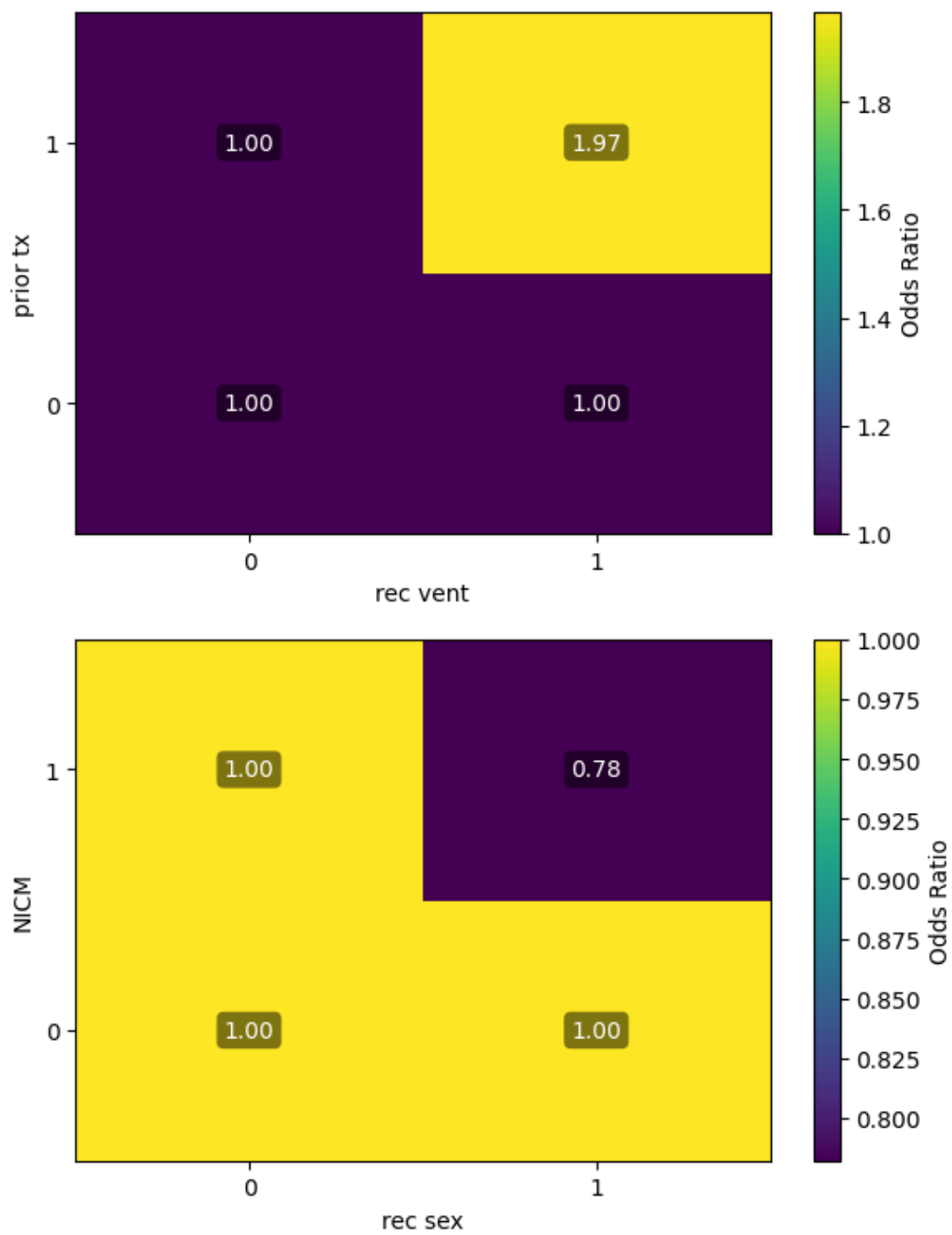
```
[184]: nomogram_params = {
        'n_steps': 15,
        'sd_scale': 2,
        'method': partial_response_method,
        'device': device,
        'categorical_threshold': 15,
        'subtract_univariate': True,
        'return_fig': True,
        'use_odds_ratio': True
    }

    nomogram_results = nomogram(
        lasso_results,
        X_train_tensor,
        X_train.median().values,
        X_train.std().values,
        blackbox_model,
        **nomogram_params
    )
    nomogram_main_mlp = nomogram_results[6]
    nomogram_non_mixed_mlp = nomogram_results[7]
```

Nomogram of univariate and mixed bivariate partial responses (Dirac)



Nomogram of non-mixed bivariate partial responses (Dirac)



1.8 8. Train Partial Response Network (PRN)

Now we train a Partial Response Network based on the selected features. The PRN is a type of neural network that incorporates the interpretability of partial responses while maintaining the predictive power of the original black box model.

From the results of the lasso feature selection, we create an input mask for the PRN. The mask is a binary matrix that determines which input features connect to which hidden nodes (subnets) in the network. This enforces the network structure based on selected features.

The PRN uses a subnet architecture, where groups of hidden nodes are dedicated to specific features or feature interactions:

- Univariate subnets: Handle individual feature effects
- Bivariate subnets: Capture interactions between two features
- The number of nodes per subnet is controlled by the `subnet_nodes` parameter in `mlpmask_pytorch` (default 5)

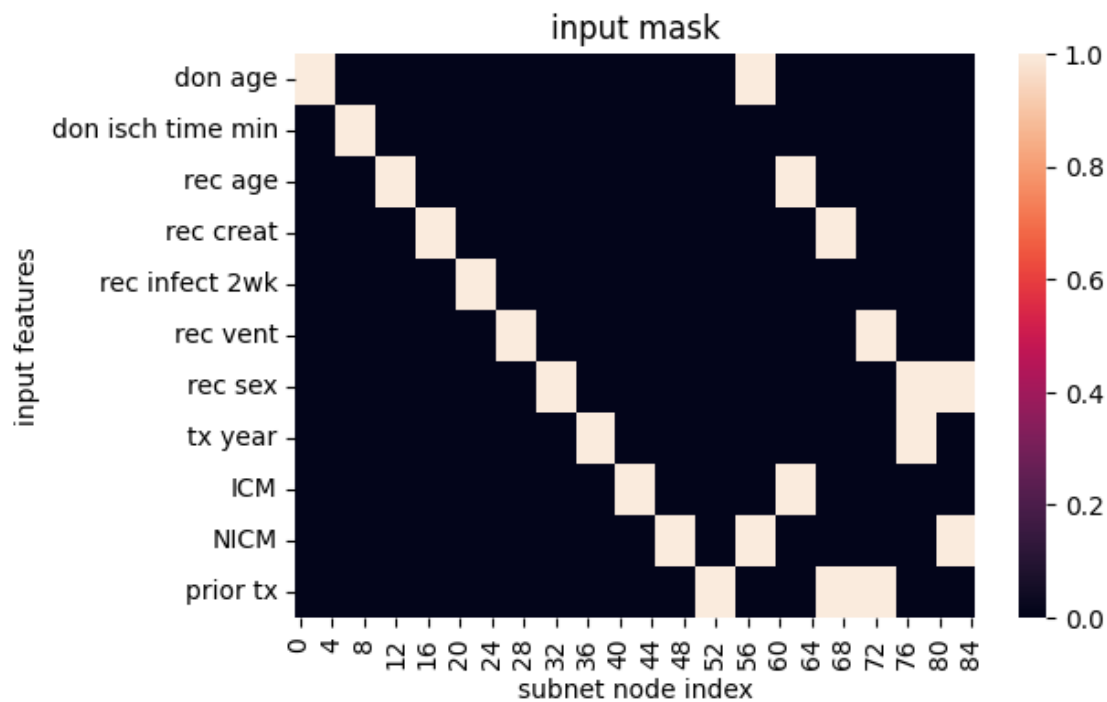
This gives us a resulting network structure as follows:

- Input layer: Original features
- Hidden layer: Structured into subnets based on the mask
- Output layer: Single node for binary classification

The resulting PRN maintains interpretability because each subnet corresponds to a specific feature or interaction. Therefore, the contribution of each feature/interaction can be analyzed separately, and the overall prediction is a sum of these interpretable components.

```
[185]: mask, n_features = lasso_results.get_mask()
```

```
Selected features: ['don age', 'don isch time min', 'rec age', 'rec creat', 'rec  
infect 2wk', 'rec vent', 'rec sex', 'tx year', 'ICM', 'NICM', 'don age : NICM',  
'rec age : ICM', 'rec creat : prior tx', 'rec vent : prior tx', 'rec sex : tx  
year', 'rec sex : NICM']
```



```
[186]: prn_hyperparameters = {
    'n_hidden': n_features,
    'mask': mask,
    'subnet_nodes': 5,
    'iter': 10000,
    'lr': 0.05,
    'weight_decay': 0.00001,
    'tolerance': 0.0001,
    'patience': 100,
    'device': device,
    'seed': random_seed
}

partial_response_network = train_maskedmlp(X_train_normalized, y_train,
↪X_test_normalized, y_test, **prn_hyperparameters)
```

```
Epoch 0, Training loss 0.7589221596717834, Test loss 0.49311310052871704
Epoch 1, Training loss 0.5120136737823486, Test loss 0.3688454329967499
Epoch 2, Training loss 0.3951898217201233, Test loss 0.3414647877216339
Epoch 3, Training loss 0.3761099874973297, Test loss 0.3645998239517212
Epoch 4, Training loss 0.4027462303638458, Test loss 0.3823683559894562
Epoch 5, Training loss 0.41778454184532166, Test loss 0.37718379497528076
Epoch 6, Training loss 0.4060332775115967, Test loss 0.35630738735198975
Epoch 7, Training loss 0.37901613116264343, Test loss 0.3375282883644104
```

Epoch 8, Training loss 0.3605685234069824, Test loss 0.3391048014163971
Epoch 9, Training loss 0.37324315309524536, Test loss 0.34916889667510986
Epoch 10, Training loss 0.39455923438072205, Test loss 0.34255167841911316
Epoch 11, Training loss 0.38422900438308716, Test loss 0.33557823300361633
Epoch 12, Training loss 0.363526850938797, Test loss 0.34413832426071167
Epoch 13, Training loss 0.3596554696559906, Test loss 0.3607143461704254
Epoch 14, Training loss 0.36934658885002136, Test loss 0.3715267479419708
Epoch 15, Training loss 0.3772420883178711, Test loss 0.3702763319015503
Epoch 16, Training loss 0.37534502148628235, Test loss 0.35901394486427307
Epoch 17, Training loss 0.3658924996852875, Test loss 0.3451261818408966
Epoch 18, Training loss 0.35761725902557373, Test loss 0.33692851662635803
Epoch 19, Training loss 0.35879626870155334, Test loss 0.3362906277179718
Epoch 20, Training loss 0.3665464520454407, Test loss 0.3367997109889984
Epoch 21, Training loss 0.36792564392089844, Test loss 0.336134672164917
Epoch 22, Training loss 0.36105993390083313, Test loss 0.33866822719573975
Epoch 23, Training loss 0.3563327193260193, Test loss 0.3449908196926117
Epoch 24, Training loss 0.35829785466194153, Test loss 0.35041600465774536
Epoch 25, Training loss 0.36234331130981445, Test loss 0.3508698344230652
Epoch 26, Training loss 0.3631756603717804, Test loss 0.34630003571510315
Epoch 27, Training loss 0.36010852456092834, Test loss 0.34015730023384094
Epoch 28, Training loss 0.3566186726093292, Test loss 0.3364611864089966
Epoch 29, Training loss 0.3565061092376709, Test loss 0.3359968662261963
Epoch 30, Training loss 0.3591974973678589, Test loss 0.33610138297080994
Epoch 31, Training loss 0.3601433038711548, Test loss 0.3357279300689697
Epoch 32, Training loss 0.3578721284866333, Test loss 0.3368571996688843
Epoch 33, Training loss 0.3558133542537689, Test loss 0.34012606739997864
Epoch 34, Training loss 0.35626456141471863, Test loss 0.34335675835609436
Epoch 35, Training loss 0.3577866852283478, Test loss 0.3441793620586395
Epoch 36, Training loss 0.3581071197986603, Test loss 0.34221845865249634
Epoch 37, Training loss 0.35687145590782166, Test loss 0.3390498161315918
Epoch 38, Training loss 0.3556290566921234, Test loss 0.33661001920700073
Epoch 39, Training loss 0.35579967498779297, Test loss 0.33556604385375977
Epoch 40, Training loss 0.3568066954612732, Test loss 0.3354426920413971
Epoch 41, Training loss 0.35694417357444763, Test loss 0.33609795570373535
Epoch 42, Training loss 0.35601410269737244, Test loss 0.3377932608127594
Epoch 43, Training loss 0.3553842306137085, Test loss 0.3399977385997772
Epoch 44, Training loss 0.3557048738002777, Test loss 0.34139660000801086
Epoch 45, Training loss 0.35622915625572205, Test loss 0.34109556674957275
Epoch 46, Training loss 0.35612767934799194, Test loss 0.33939242362976074
Epoch 47, Training loss 0.35553649067878723, Test loss 0.3374174237251282
Epoch 48, Training loss 0.355223149061203, Test loss 0.33611997961997986
Epoch 49, Training loss 0.3554917275905609, Test loss 0.335671991109848
Epoch 50, Training loss 0.35577985644340515, Test loss 0.33588069677352905
Epoch 51, Training loss 0.3555763363838196, Test loss 0.3367072641849518
Epoch 52, Training loss 0.3551930785179138, Test loss 0.3379884362220764
Epoch 53, Training loss 0.3551499843597412, Test loss 0.33911481499671936
Epoch 54, Training loss 0.3553740084171295, Test loss 0.33942729234695435
Epoch 55, Training loss 0.35543662309646606, Test loss 0.3388109505176544

Epoch 56, Training loss 0.3552209138870239, Test loss 0.3377563953399658
Epoch 57, Training loss 0.3550276756286621, Test loss 0.3368835151195526
Epoch 58, Training loss 0.35508570075035095, Test loss 0.336500883102417
Epoch 59, Training loss 0.3552170991897583, Test loss 0.3366357088088989
Epoch 60, Training loss 0.3551594614982605, Test loss 0.3372410833835602
Epoch 61, Training loss 0.3549908697605133, Test loss 0.33812862634658813
Epoch 62, Training loss 0.35494962334632874, Test loss 0.33887284994125366
Epoch 63, Training loss 0.3550352454185486, Test loss 0.33905237913131714
Epoch 64, Training loss 0.3550584018230438, Test loss 0.33859819173812866
Epoch 65, Training loss 0.3549613952636719, Test loss 0.33783072233200073
Epoch 66, Training loss 0.354881227016449, Test loss 0.3371700346469879
Epoch 67, Training loss 0.35490915179252625, Test loss 0.3368609845638275
Epoch 68, Training loss 0.35495230555534363, Test loss 0.33694902062416077
Epoch 69, Training loss 0.35490915179252625, Test loss 0.3373575508594513
Epoch 70, Training loss 0.3548373281955719, Test loss 0.3378826975822449
Epoch 71, Training loss 0.3548317849636078, Test loss 0.3382285535335541
Epoch 72, Training loss 0.3548645079135895, Test loss 0.33819007873535156
Epoch 73, Training loss 0.3548530340194702, Test loss 0.33781105279922485
Epoch 74, Training loss 0.35480207204818726, Test loss 0.33733317255973816
Epoch 75, Training loss 0.35478079319000244, Test loss 0.3370034992694855
Epoch 76, Training loss 0.35479944944381714, Test loss 0.3369466960430145
Epoch 77, Training loss 0.35480061173439026, Test loss 0.33716338872909546
Epoch 78, Training loss 0.35476696491241455, Test loss 0.3375491499900818
Epoch 79, Training loss 0.3547438681125641, Test loss 0.3379121422767639
Epoch 80, Training loss 0.3547513484954834, Test loss 0.3380621671676636
Epoch 81, Training loss 0.35475486516952515, Test loss 0.3379407525062561
Epoch 82, Training loss 0.35473358631134033, Test loss 0.33765706419944763
Epoch 83, Training loss 0.35471320152282715, Test loss 0.3373945355415344
Epoch 84, Training loss 0.35471412539482117, Test loss 0.3372906446456909
Epoch 85, Training loss 0.3547171354293823, Test loss 0.3373848795890808
Epoch 86, Training loss 0.3547031581401825, Test loss 0.33761951327323914
Epoch 87, Training loss 0.3546869456768036, Test loss 0.33786386251449585
Epoch 88, Training loss 0.354684978723526, Test loss 0.3379775881767273
Epoch 89, Training loss 0.35468605160713196, Test loss 0.3379001319408417
Epoch 90, Training loss 0.35467594861984253, Test loss 0.3376897871494293
Epoch 91, Training loss 0.3546631634235382, Test loss 0.3374738395214081
Epoch 92, Training loss 0.3546598553657532, Test loss 0.33736228942871094
Epoch 93, Training loss 0.35465943813323975, Test loss 0.33739563822746277
Epoch 94, Training loss 0.3546515703201294, Test loss 0.3375377357006073
Epoch 95, Training loss 0.35464176535606384, Test loss 0.3376973867416382
Epoch 96, Training loss 0.35463839769363403, Test loss 0.337776780128479
Epoch 97, Training loss 0.3546368479728699, Test loss 0.3377327024936676
Epoch 98, Training loss 0.3546302020549774, Test loss 0.33760273456573486
Epoch 99, Training loss 0.35462236404418945, Test loss 0.3374728560447693
Epoch 100, Training loss 0.35461893677711487, Test loss 0.33741894364356995
Epoch 101, Training loss 0.3546164631843567, Test loss 0.3374667465686798
Epoch 102, Training loss 0.354610413312912, Test loss 0.3375844955444336
Epoch 103, Training loss 0.354604035615921, Test loss 0.3377021849155426

Epoch 104, Training loss 0.3546007573604584, Test loss 0.33775246143341064
Epoch 105, Training loss 0.35459768772125244, Test loss 0.33771365880966187
Epoch 106, Training loss 0.3545922338962555, Test loss 0.33762097358703613
Epoch 107, Training loss 0.35458695888519287, Test loss 0.3375379145145416
Epoch 108, Training loss 0.3545837700366974, Test loss 0.3375139832496643
Epoch 109, Training loss 0.3545803427696228, Test loss 0.33755776286125183
Epoch 110, Training loss 0.35457536578178406, Test loss 0.3376363217830658
Epoch 111, Training loss 0.3545708656311035, Test loss 0.3376966714859009
Epoch 112, Training loss 0.3545677363872528, Test loss 0.3377000391483307
Epoch 113, Training loss 0.3545640707015991, Test loss 0.33764657378196716
Epoch 114, Training loss 0.35455945134162903, Test loss 0.3375726044178009
Epoch 115, Training loss 0.35455557703971863, Test loss 0.3375234305858612
Epoch 116, Training loss 0.35455232858657837, Test loss 0.3375243842601776
Epoch 117, Training loss 0.3545486032962799, Test loss 0.33756789565086365
Epoch 118, Training loss 0.35454443097114563, Test loss 0.33762097358703613
Epoch 119, Training loss 0.3545408546924591, Test loss 0.3376471996307373
Epoch 120, Training loss 0.3545375466346741, Test loss 0.3376310169696808
Epoch 121, Training loss 0.3545338213443756, Test loss 0.3375866413116455
Epoch 122, Training loss 0.3545299768447876, Test loss 0.33754581212997437
Epoch 123, Training loss 0.3545266389846802, Test loss 0.33753496408462524
Epoch 124, Training loss 0.35452327132225037, Test loss 0.3375589847564697
Epoch 125, Training loss 0.35451963543891907, Test loss 0.33759981393814087
Epoch 126, Training loss 0.35451608896255493, Test loss 0.3376295864582062
Epoch 127, Training loss 0.35451290011405945, Test loss 0.3376298248767853
Epoch 128, Training loss 0.35450953245162964, Test loss 0.33760324120521545
Epoch 129, Training loss 0.3545059859752655, Test loss 0.33757030963897705
Epoch 130, Training loss 0.35450273752212524, Test loss 0.3375532627105713
Epoch 131, Training loss 0.3544995188713074, Test loss 0.33756136894226074
Epoch 132, Training loss 0.35449621081352234, Test loss 0.3375859260559082
Epoch 133, Training loss 0.3544928729534149, Test loss 0.33760741353034973
Epoch 134, Training loss 0.3544897139072418, Test loss 0.33761003613471985
Epoch 135, Training loss 0.3544866144657135, Test loss 0.33759215474128723
Epoch 136, Training loss 0.35448330640792847, Test loss 0.3375668227672577
Epoch 137, Training loss 0.354480117559433, Test loss 0.3375508785247803
Epoch 138, Training loss 0.35447704792022705, Test loss 0.33755335211753845
Epoch 139, Training loss 0.35447388887405396, Test loss 0.3375697433948517
Epoch 140, Training loss 0.35447070002555847, Test loss 0.3375864028930664
Early stopping! Epoch 140
Best model at epoch 40

```
[187]: # Evaluate PRN on training data
y_pred_train_prn = partial_response_network.predict(X_train_tensor,
↪device=device)

results_prn_train = evaluate_model_performance(y_train, y_pred_train_prn,
↪y_train, title="PRN - train data")
```

```
# Evaluate PRN on test data and compare to MLP (blackbox)
y_pred_test_prn = partial_response_network.predict(X_test_tensor, device=device)

results_prn_mlp_test = compare_model_performance(y_test, y_pred_test_blackbox,
↪y_pred_test_prn, y_train=y_train, model_names=("MLP", "PRN"), title="MLP-PRN
↪test comparison")
```

---- Model Performance Metrics PRN - train data ----

Threshold (prevalence) set to: 0.123

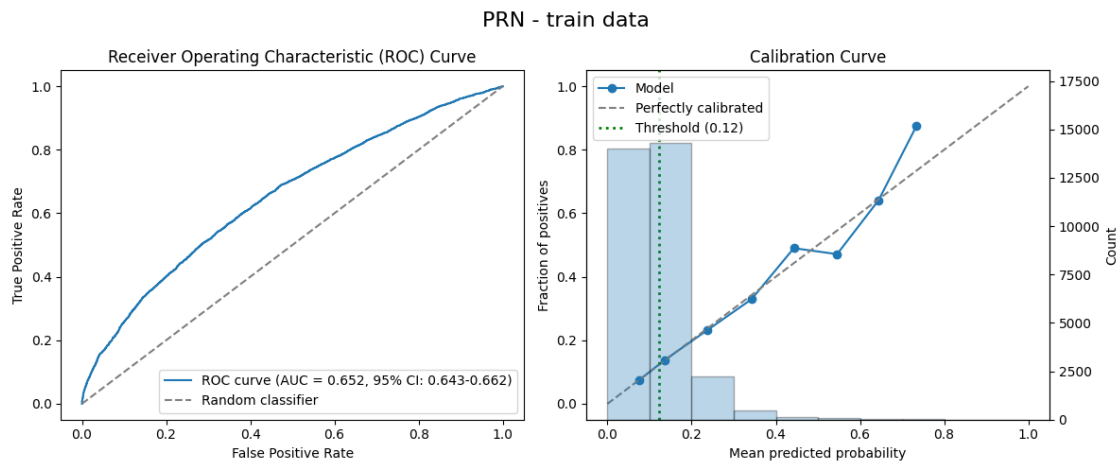
AUROC: 0.652 (95% CI: 0.643-0.662) (Area Under the Receiver Operating Characteristic curve)

Accuracy: 0.646 (Proportion of correct predictions)

Precision: 0.187 (Proportion of true positives among positive predictions, aka PPV)

Recall: 0.559 (Proportion of true positives among actual positives, aka sensitivity)

F1 Score: 0.280 (Harmonic mean of precision and recall)



----- Model Performance Comparison -----

Threshold (prevalence) set to: 0.123

MLP:

AUROC: 0.606 (95% CI: 0.583-0.630)

Accuracy: 0.801

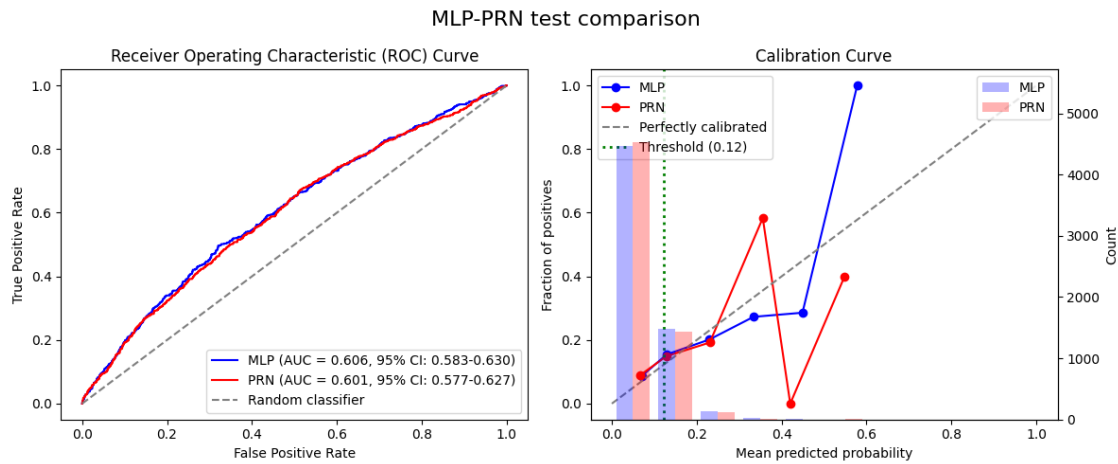
Precision: 0.177

Recall: 0.235

F1 Score: 0.202

PRN:

AUROC: 0.601 (95% CI: 0.577-0.627)
 Accuracy: 0.807
 Precision: 0.179
 Recall: 0.223
 F1 Score: 0.198



1.9 9. LASSO on PRN

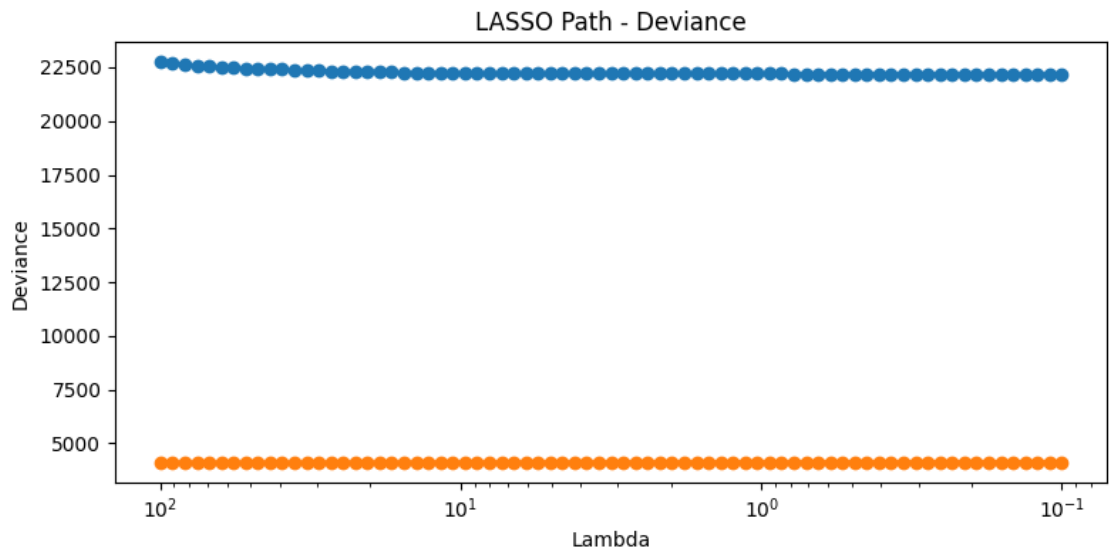
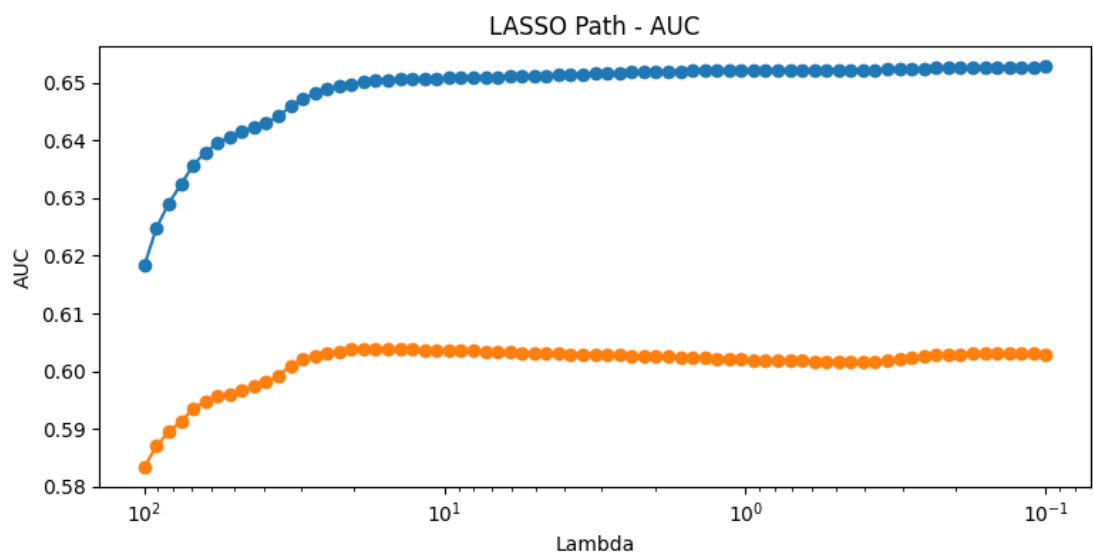
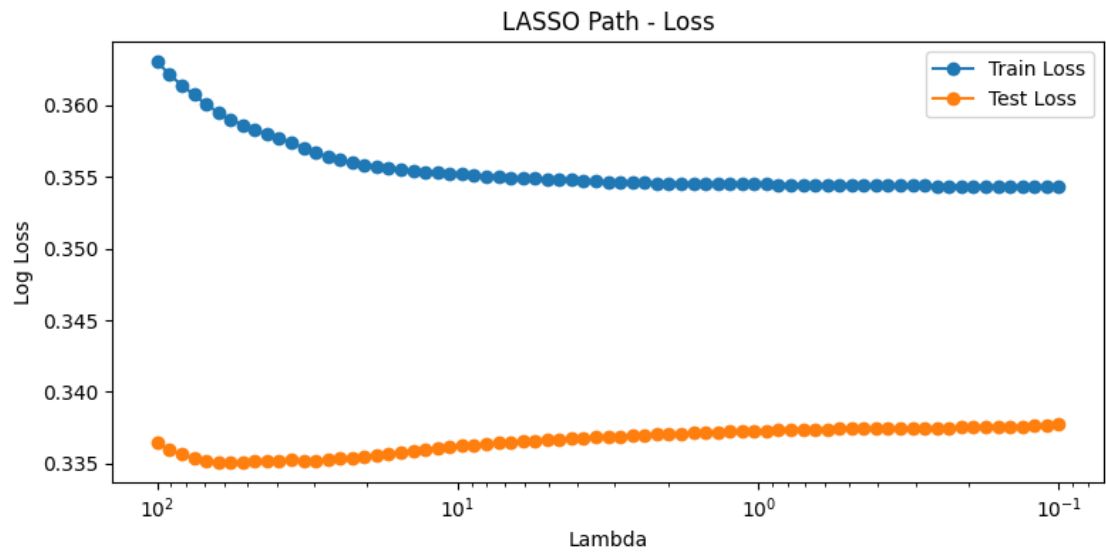
We perform LASSO again, this time on the Partial Response Network to isolate the most important features and interactions.

```
[188]: partial_responses_params_prn = {
    'x_train': X_train_tensor,
    'method': partial_response_method,
    'device': device,
    'batch_size': 256,
    'max_workers': max_workers
}

partial_responses_train_prn = partial_responses(X_train_tensor,
    ↪ partial_response_network, **partial_responses_params_prn)
partial_responses_test_prn = partial_responses(X_test_tensor,
    ↪ partial_response_network, **partial_responses_params_prn)

[189]: lasso_results_prn = lasso(
    partial_responses_train_prn,
    partial_responses_test_prn,
    y_train,
    y_test,
    feature_names=feature_names,
```

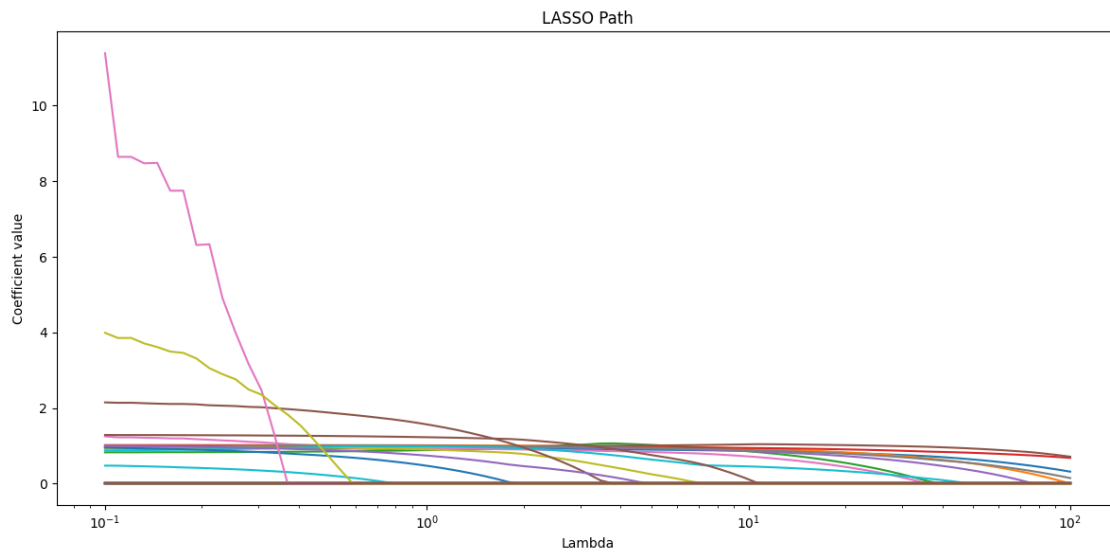
```
nlambda=75,  
min_lambda=0.1,  
max_lambda=100,  
batch_size=2,  
seed=random_seed  
)
```



<IPython.core.display.HTML object>

All fits converged successfully.

```
[190]: lasso_results_prn.plot_lambda_path()
```



```
[191]: lasso_results_prn.select_lambda_max_test_auc()

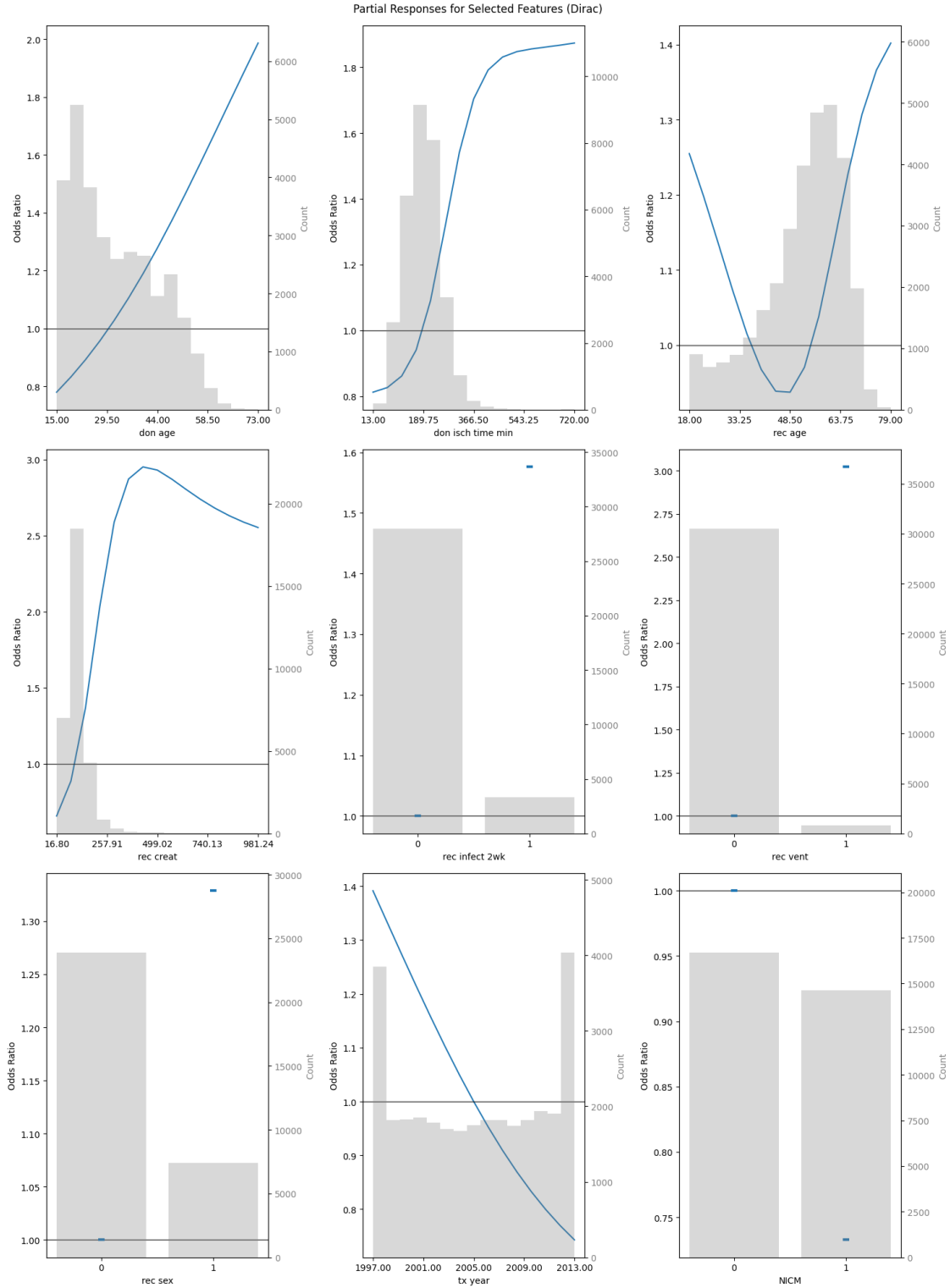
# Alternatively, manually select lambda as needed
# lasso_results_prn.select_lambda(20)

lasso_results_prn.plot_feature_importance()
```

Selected lambda index: 21

Selected lambda value: 14.0815

Corresponding test AUC: 0.6038



Finally, we generate a nomogram for our PRN, based on the features selected with LASSO. The nomogram provides a visual tool that can be used to understand the model's decisions.

```

[193]: %%capture
# ^ Supress nomogram plot output, will display side-by-side instead

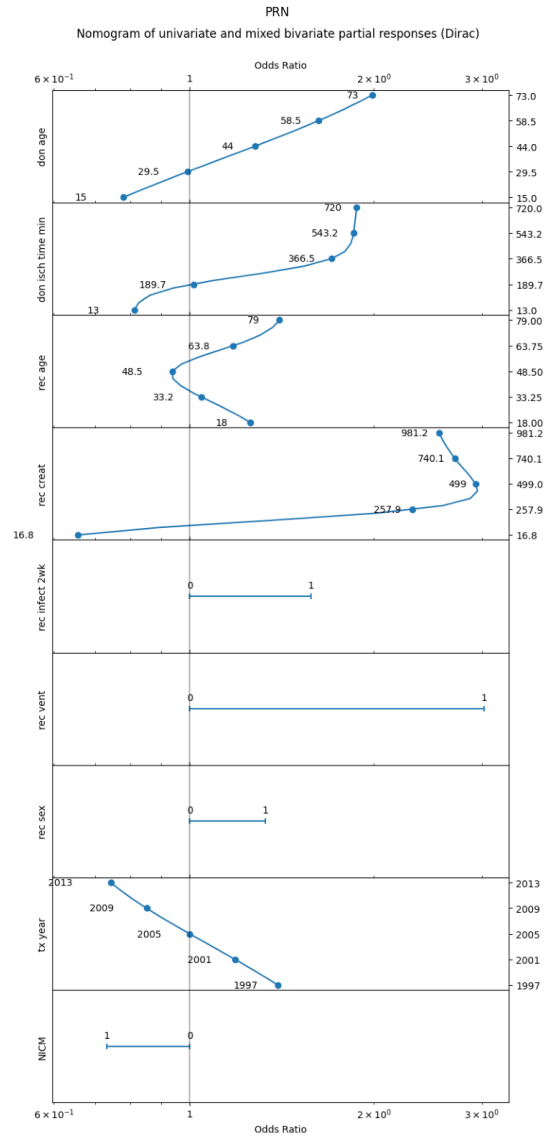
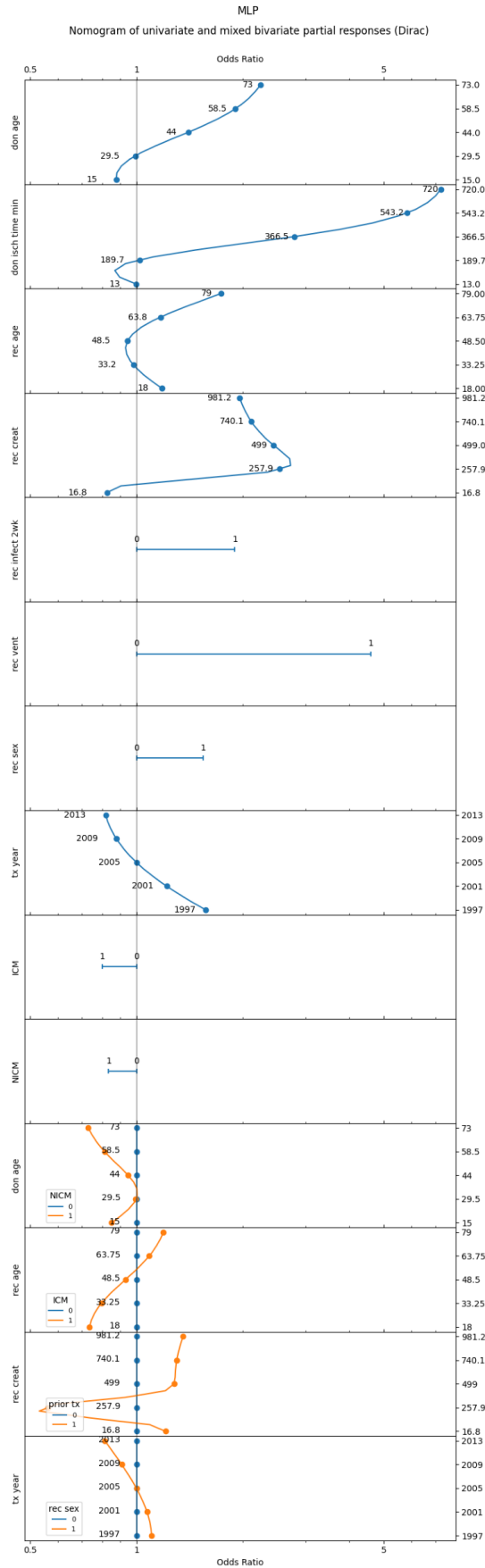
nomogram_params_prn = {
    'n_steps': 15,
    'sd_scale': 2,
    'method': partial_response_method,
    'device': device,
    'categorical_threshold': 15,
    'subtract_univariate': True,
    'show_fig': False,
    'return_fig': True,
    'use_odds_ratio': True
}

nomogram_results_prn = nomogram(
    lasso_results_prn,
    X_train_tensor,
    X_train.median().values,
    X_train.std().values,
    partial_response_network,
    **nomogram_params_prn
)

nomogram_main_prn = nomogram_results_prn[6]
nomogram_non_mixed_prn = nomogram_results_prn[7]

[194]: display_nomograms_side_by_side(nomogram_main_mlp, nomogram_non_mixed_mlp,
                                     nomogram_main_prn, nomogram_non_mixed_prn)

```



1.11 11. Evaluate PRN-LASSO Model

Finally, we evaluate the simplified model (multivariate logistic regression with LASSO regularization) trained on the partial responses of the PRN, based on the selected lambda.

This notebook demonstrates the PRiSM method, transforming a black box model into an interpretable one without sacrificing predictive performance. This approach is particularly valuable when understanding the model's decision-making process is crucial for building trust and gaining insights in clinical decision-making scenarios like predicting one-year mortality after heart transplantation.

The final PRN-LASSO model provides both high predictive accuracy and interpretability, allowing us to understand how different features contribute to the predictions of post-transplant mortality risk.

```
[195]: prn_lasso = lasso_results_prn.get_selected_model()
```

Logistic regression model for Lambda index 21 (14.08) selected.

```
[196]: # Evaluate PRN-LASSO on training data
y_pred_train_prn_lasso = prn_lasso.predict_proba(partial_responses_train_prn)[:
    ↪, 1]

results_prn_lasso_train = evaluate_model_performance(y_train,
    ↪y_pred_train_prn_lasso, y_train, title="PRN LASSO - train data")

# Evaluate PRN-LASSO on test data and compare to PRN
y_pred_test_prn_lasso = prn_lasso.predict_proba(partial_responses_test_prn)[:
    ↪1]

results_prn_prn_lasso_test = compare_model_performance(y_test, y_pred_test_prn,
    ↪y_pred_test_prn_lasso, y_train=y_train, model_names=("PRN", "PRN LASSO"),
    ↪title="PRN - PRN LASSO test comparison")

# Compare PRN-LASSO and original MLP (blackbox) model
results_prn_mlp_test = compare_model_performance(y_test, y_pred_test_blackbox,
    ↪y_pred_test_prn_lasso, y_train=y_train, model_names=("MLP", "PRN LASSO"),
    ↪title="MLP - PRN LASSO test comparison")
```

---- Model Performance Metrics PRN LASSO - train data ----

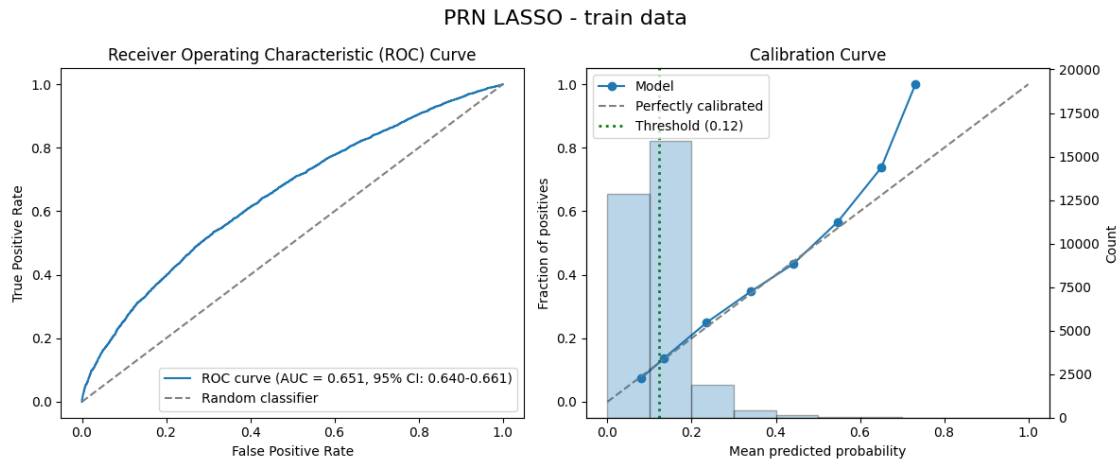
Threshold (prevalence) set to: 0.123

AUROC: 0.651 (95% CI: 0.640-0.661) (Area Under the Receiver Operating Characteristic curve)

Accuracy: 0.643 (Proportion of correct predictions)

Precision: 0.185 (Proportion of true positives among positive predictions, aka PPV)

Recall: 0.560 (Proportion of true positives among actual positives, aka sensitivity)
F1 Score: 0.278 (Harmonic mean of precision and recall)



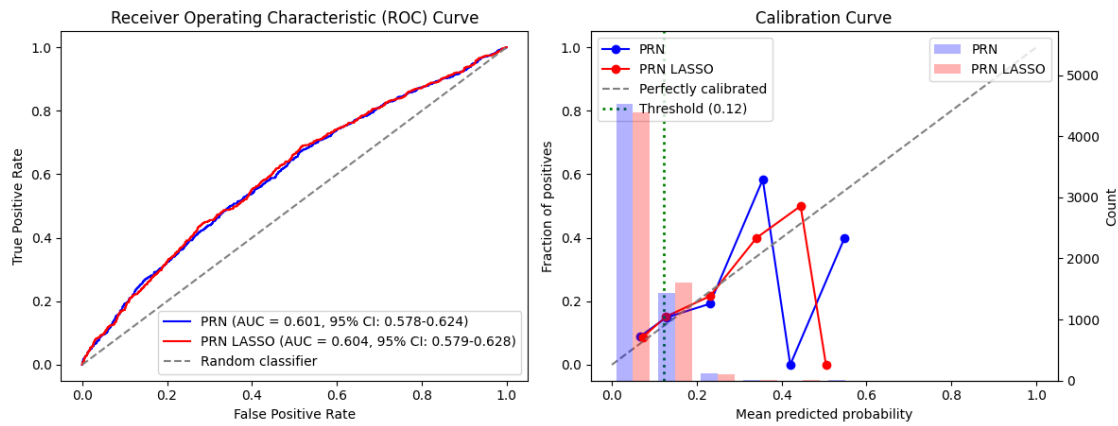
----- Model Performance Comparison -----

Threshold (prevalence) set to: 0.123

PRN:
AUROC: 0.601 (95% CI: 0.578-0.624)
Accuracy: 0.807
Precision: 0.179
Recall: 0.223
F1 Score: 0.198

PRN LASSO:
AUROC: 0.604 (95% CI: 0.579-0.628)
Accuracy: 0.805
Precision: 0.175
Recall: 0.221
F1 Score: 0.195

PRN - PRN LASSO test comparison



----- Model Performance Comparison -----
Threshold (prevalence) set to: 0.123

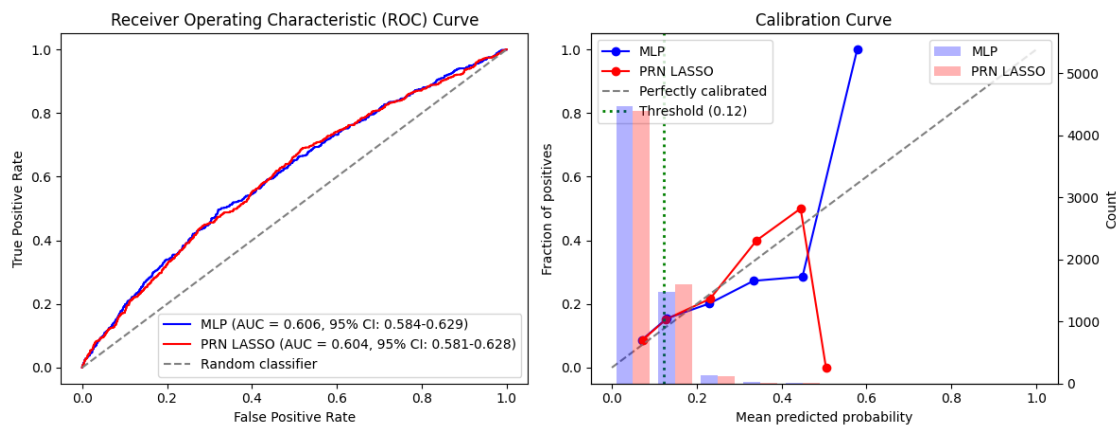
MLP:

AUROC: 0.606 (95% CI: 0.584-0.629)
Accuracy: 0.801
Precision: 0.177
Recall: 0.235
F1 Score: 0.202

PRN LASSO:

AUROC: 0.604 (95% CI: 0.581-0.628)
Accuracy: 0.805
Precision: 0.175
Recall: 0.221
F1 Score: 0.195

MLP - PRN LASSO test comparison



```
[197]: # Calculate partial responses on validation dataset
partial_responses_val_prn = partial_responses(X_val_tensor,
↳partial_response_network, **partial_responses_params_prn)

[198]: # Evaluate PRN-LASSO on validation dataset and compare to PRN
y_pred_val_prn = partial_response_network.predict(X_val_tensor, device=device)
y_pred_val_prn_lasso = prn_lasso.predict_proba(partial_responses_val_prn)[: , 1]

results_prn_prn_lasso_test = compare_model_performance(y_val, y_pred_val_prn,
↳y_pred_val_prn_lasso, y_train=y_train, model_names=("PRN", "PRN LASSO"),
↳title="PRN - PRN LASSO validation comparison")
```

----- Model Performance Comparison -----
Threshold (prevalence) set to: 0.123

PRN:
AUROC: 0.609 (95% CI: 0.583-0.634)
Accuracy: 0.543
Precision: 0.129
Recall: 0.626
F1 Score: 0.214

PRN LASSO:
AUROC: 0.604 (95% CI: 0.578-0.632)
Accuracy: 0.660
Precision: 0.135
Recall: 0.448
F1 Score: 0.207

